Tarea 2: Path Tracing con OpenMP y Numba

Declaracion de uso de IA

Se uso un asistente de IA para revisar la pauta y clases, implementar las versiones OpenMP y Numba, automatizar experimentos, revisar cobertura de resultados y generar este informe. Los resultados numericos provienen de `results/final/local/results.csv` y `results/final/cluster/results.csv`.

Entorno de ejecucion

Maquina	Procesador	Cores fisicos	Threads logicos	Threads/core	Max MHz
Local	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz	6	12	2	No informado
Cluster	Intel(R) Xeon(R) Silver 4210R CPU @ 2.40GHz	20	40	2	3200.0000

Cobertura de resultados

Ambas corridas corresponden a la matriz completa con `--full --reps 1`: dos escenas, tres configuraciones y p en {1, 2, 4, 8}.

Maquina	Filas	Serial	Scheduling	OpenMP cmp.	Numba cmp.
Local	148	12	88	24	24
Cluster	148	12	88	24	24

Configuraciones

Config.	Resolucion	S ray tracing	N path tracing
small	400 x 300	2	32
medium	800 x 600	4	128
large	1600 x 1200	4	256

Implementacion

`omp_pt.cpp` paraleliza el loop de pixeles con `#pragma omp parallel for`. `omp_pt_sched.cpp` usa `schedule(runtime)` y `omp_set_schedule` para probar `static`, `dynamic` y `guided` con distintos chunks. Cada pixel es independiente y escribe una posicion unica en la imagen, por lo que la paralelizacion sigue el patron map.

`numba_pt.py` implementa la misma logica con `@njit(parallel=True)` y `prange`. El script realiza un warm-up minimo antes de medir para excluir el costo de compilacion JIT inicial de Numba.

Resultados seriales y crecimiento

La tabla compara `serial_pt.cpp`; el cociente es `Cluster/Local`, por lo que valores mayores a 1 indican que el cluster fue mas lento en esa medicion.

Escena	Config.	Local Ts	Cluster Ts	Cluster/Local
scene.txt	small	4.075	6.443	1.58x
scene.txt	medium	64.690	100.985	1.56x
scene.txt	large	533.884	811.762	1.52x
scene_many.txt	small	6.177	11.424	1.85x
scene_many.txt	medium	98.010	175.809	1.79x
scene_many.txt	large	782.014	1406.140	1.80x

Crecimiento Local

Escena	small -> medium	medium -> large
scene.txt	15.87x	8.25x
scene_many.txt	15.87x	7.98x

Crecimiento Cluster

Escena	small -> medium	medium -> large
scene.txt	15.67x	8.04x
scene_many.txt	15.39x	8.00x

El crecimiento es consistente con el costo esperado O(WHNDK). De `small` a `medium` se cuadruplica la resolucion y N pasa de 32 a 128, por lo que se espera cerca de 16x. De `medium` a `large` la resolucion vuelve a crecer 4x y N se duplica, por lo que se espera cerca de 8x.

Barrido de scheduling OpenMP

Local

Escena	Mejor promedio	Tiempo prom.
scene.txt	guided(8)	34.511
scene_many.txt	dynamic(128)	53.780

Escena	p=1	p=2	p=4	p=8
scene.txt	static(32), 64.336s	guided(8), 37.323s	static(1), 21.527s	dynamic(1), 14.067s
scene_many.txt	dynamic(128), 99.200s	guided(32), 57.949s	static(1), 34.195s	dynamic(8), 22.673s

Cluster

Escena	Mejor promedio	Tiempo prom.
scene.txt	dynamic(128)	50.311
scene_many.txt	dynamic(1)	86.811

Escena	p=1	p=2	p=4	p=8
scene.txt	guided(1), 101.302s	dynamic(128), 52.347s	guided(32), 27.533s	dynamic(1), 19.556s
scene_many.txt	dynamic(1), 169.603s	guided(1), 89.833s	guided(8), 47.791s	dynamic(1), 37.343s

No aparece una politica unica para todos los casos. La carga por pixel es irregular porque algunos rayos terminan en el fondo y otros hacen mas rebotes, sombras e intersecciones; por eso `dynamic` o `guided` pueden mejorar el balance, aunque chunks demasiado pequenos agregan overhead. En la escena con 40 esferas aumenta el costo de interseccion y el balance de carga pesa mas.

OpenMP vs Numba

Para cada punto se usa como referencia el tiempo `serial_pt` de la misma escena y configuracion. Se calcula `S(p)=Ts/T(p)` y `E(p)=S(p)/p`.

Resumen a 8 threads - Local

Escena	Config.	Ts	OMP T8	OMP S8	OMP E8	Numba T8	Numba S8	Numba E8	Mejor
scene.txt	small	4.075	0.765	5.32	0.67	0.659	6.18	0.77	Numba
scene.txt	medium	64.690	15.464	4.18	0.52	12.602	5.13	0.64	Numba
scene.txt	large	533.884	112.857	4.73	0.59	102.854	5.19	0.65	Numba
scene_many.txt	small	6.177	1.251	4.94	0.62	1.298	4.76	0.59	OpenMP
scene_many.txt	medium	98.010	22.855	4.29	0.54	23.217	4.22	0.53	OpenMP
scene_many.txt	large	782.014	182.725	4.28	0.53	185.490	4.22	0.53	OpenMP

En Local, Numba fue mas rapido que OpenMP en 19 de 24 puntos comparables. El cociente promedio `T_OpenMP/T_Numba` fue 1.13x. La mayor ventaja de Numba fue 1.45x en scene.txt `large` con p=1. La menor razon fue 0.96x en scene_many.txt `small` con p=8.

Resumen a 8 threads - Cluster

Escena	Config.	Ts	OMP T8	OMP S8	OMP E8	Numba T8	Numba S8	Numba E8	Mejor
scene.txt	small	6.443	1.269	5.08	0.63	1.084	5.94	0.74	Numba
scene.txt	medium	100.985	19.699	5.13	0.64	16.977	5.95	0.74	Numba
scene.txt	large	811.762	157.145	5.17	0.65	135.203	6.00	0.75	Numba
scene_many.txt	small	11.424	2.380	4.80	0.60	2.013	5.68	0.71	Numba
scene_many.txt	medium	175.809	37.521	4.69	0.59	31.523	5.58	0.70	Numba
scene_many.txt	large	1406.140	298.016	4.72	0.59	252.487	5.57	0.70	Numba

En Cluster, Numba fue mas rapido que OpenMP en 24 de 24 puntos comparables. El cociente promedio `T_OpenMP/T_Numba` fue 1.24x. La mayor ventaja de Numba fue 1.41x en scene.txt `large` con p=1. La menor razon fue 1.15x en scene_many.txt `large` con p=4.

La eficiencia cae al aumentar p por overhead de planificacion, barreras implicitas, sincronizacion, presion de cache y saturacion de recursos compartidos. Aunque el cluster tiene mas cores fisicos, estas corridas solo usan hasta 8 threads, por lo que no aprovechan los 20 cores fisicos completos del nodo.

Comparacion entre computadores

La comparacion usa p=8 para las implementaciones paralelas. El cociente es `Cluster/Local`; valores mayores a 1 indican que el cluster demoro mas que el equipo local.

OpenMP p=8

Escena	Config.	Local T8	Cluster T8	Cluster/Local	Mas rapido
scene.txt	small	0.765	1.269	1.66x	Local
scene.txt	medium	15.464	19.699	1.27x	Local
scene.txt	large	112.857	157.145	1.39x	Local
scene_many.txt	small	1.251	2.380	1.90x	Local
scene_many.txt	medium	22.855	37.521	1.64x	Local
scene_many.txt	large	182.725	298.016	1.63x	Local

Numba p=8

Escena	Config.	Local T8	Cluster T8	Cluster/Local	Mas rapido
scene.txt	small	0.659	1.084	1.65x	Local
scene.txt	medium	12.602	16.977	1.35x	Local
scene.txt	large	102.854	135.203	1.31x	Local
scene_many.txt	small	1.298	2.013	1.55x	Local
scene_many.txt	medium	23.217	31.523	1.36x	Local
scene_many.txt	large	185.490	252.487	1.36x	Local

En estas mediciones el equipo local fue mas rapido en todos los puntos p=8, pese a tener menos cores fisicos. Esto es plausible porque la comparacion limita p a 8, no a la capacidad total del cluster; ademas, el Xeon Silver 4210R tiene menor frecuencia base y el trabajo puede depender de rendimiento por thread, cache, politicas de frecuencia, contencion del nodo y overhead del entorno. El cluster sigue siendo valido como segundo computador porque entrega otra arquitectura y otra configuracion de cores/threads para comparar.

Graficos

Los graficos siguientes fueron generados por el mismo script desde cada `results.csv`.

Local

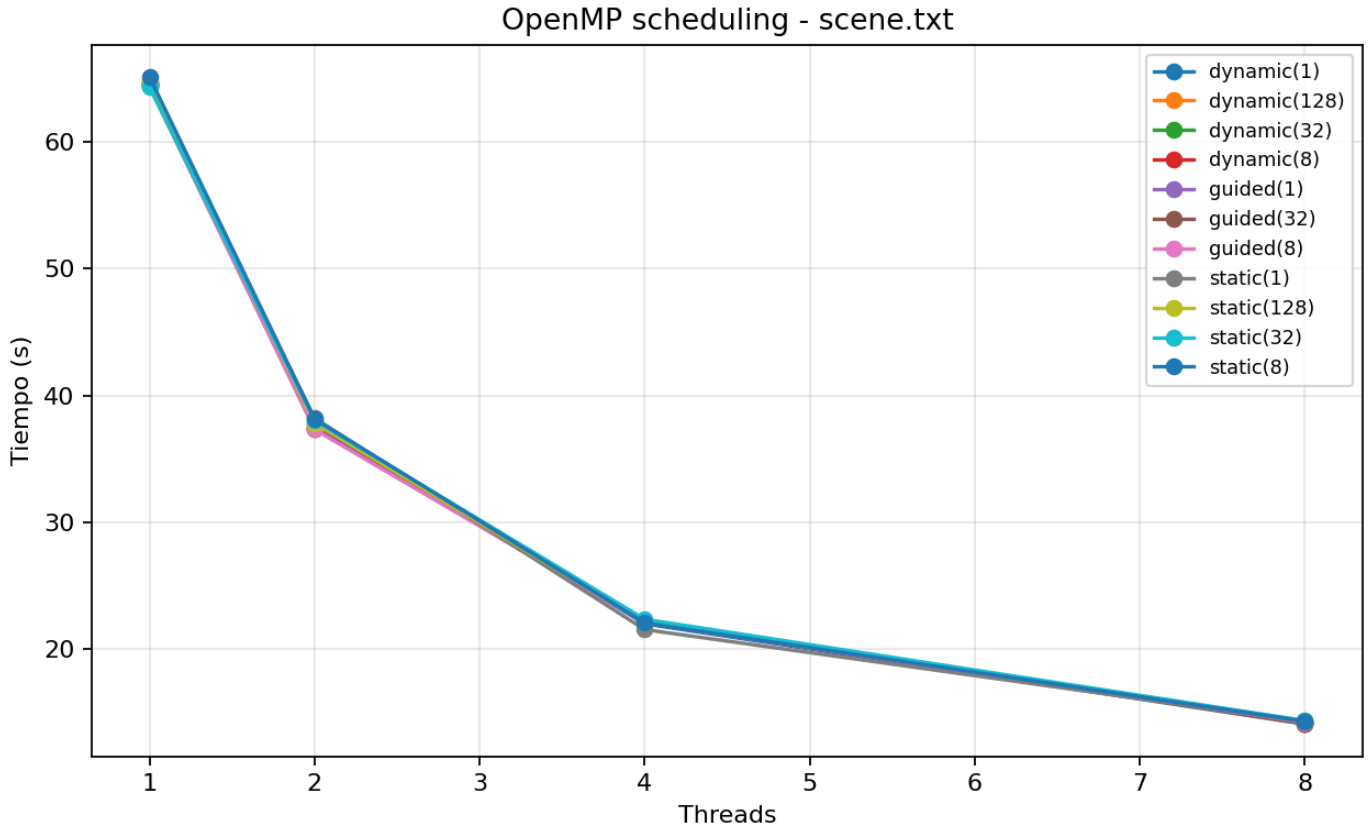

Figure 1: local: Scheduling scene.txt

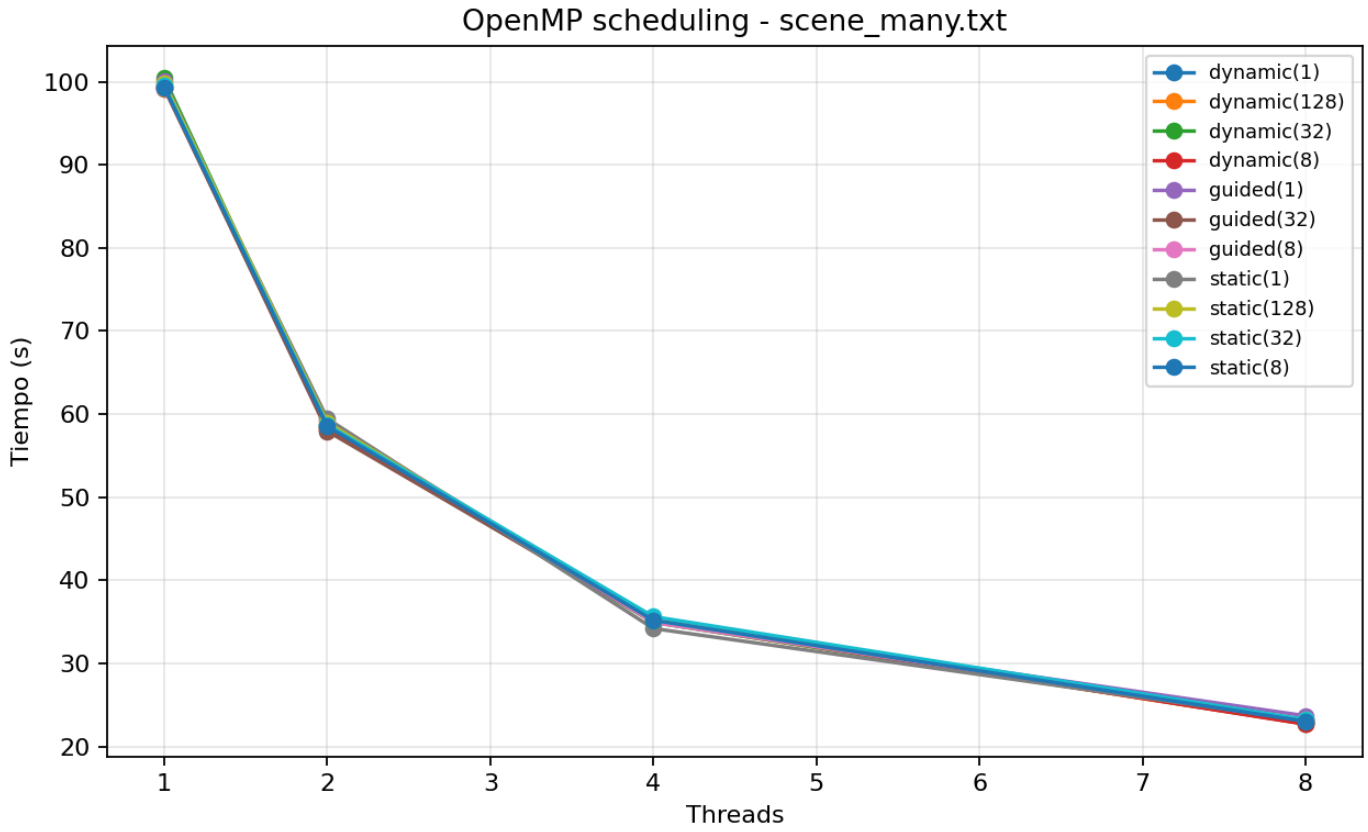

Figure 2: local: Scheduling scene_many.txt

local - scene.txt small

local - scene.txt medium

local - scene.txt large

local - scene_many.txt small

local - scene_many.txt medium

local - scene_many.txt large

Cluster

cluster - scene.txt small

cluster - scene.txt medium

cluster - scene.txt large

cluster - scene_many.txt small

cluster - scene_many.txt medium

cluster - scene_many.txt large

Archivos de respaldo

- Resultados locales: `results/final/local/results.csv`, `hardware.json`, `report.md`, `report.pdf` y `plots/`.
- Resultados cluster: `results/final/cluster/results.csv`, `hardware.json`, `report.md`, `report.pdf`, logs `slurm_2289.out` y `slurm_2289.err`, y `plots/`.

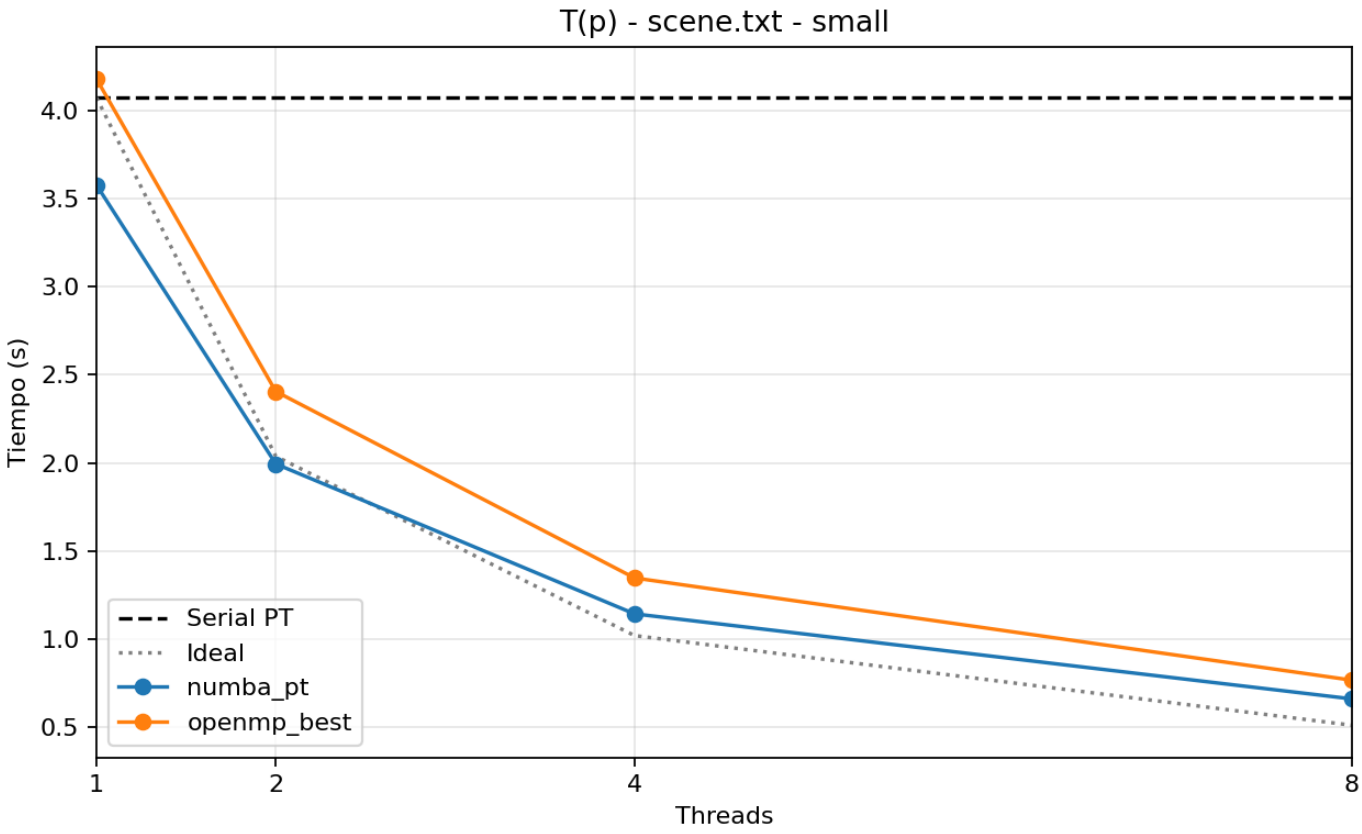

Figure 3: local: Tiempo scene.txt small

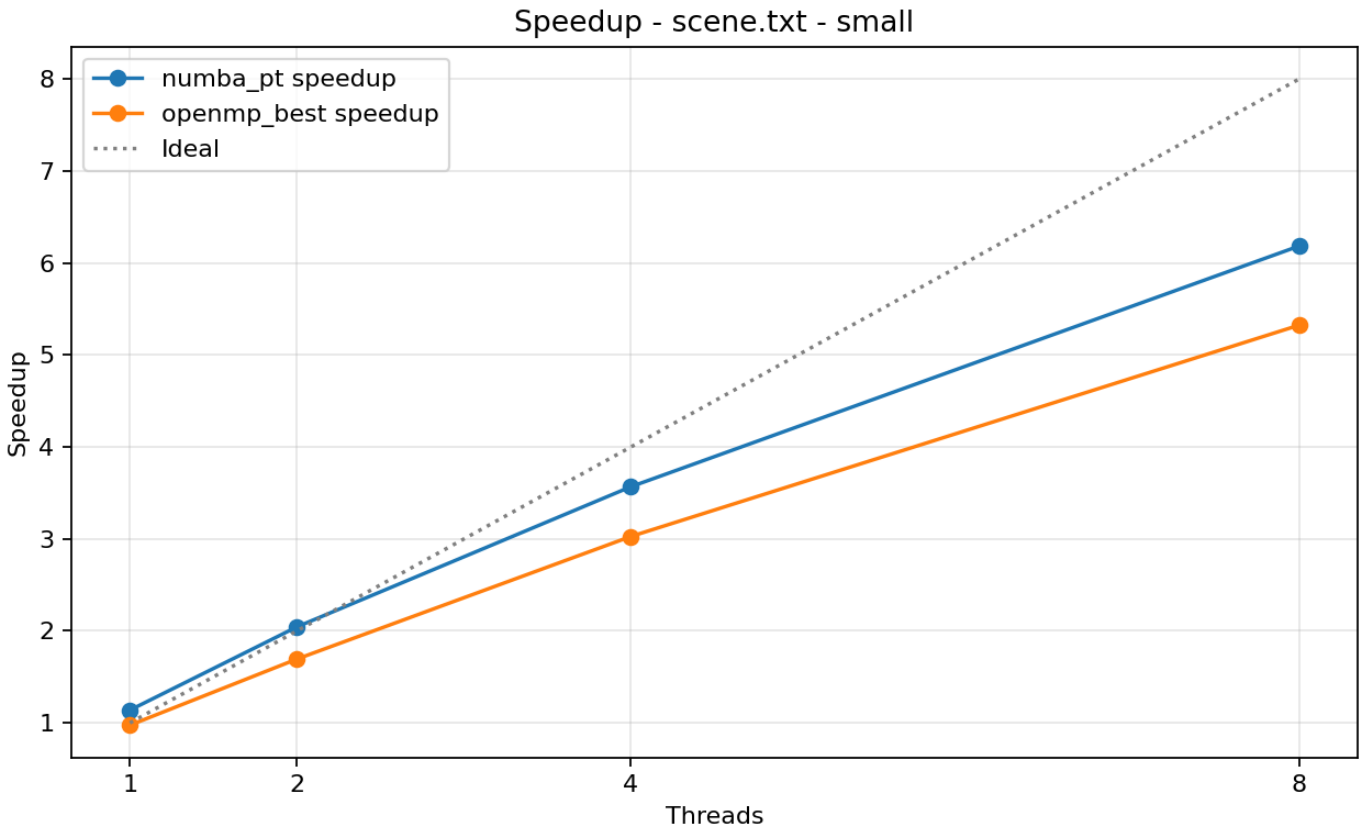

Figure 4: local: Speedup scene.txt small

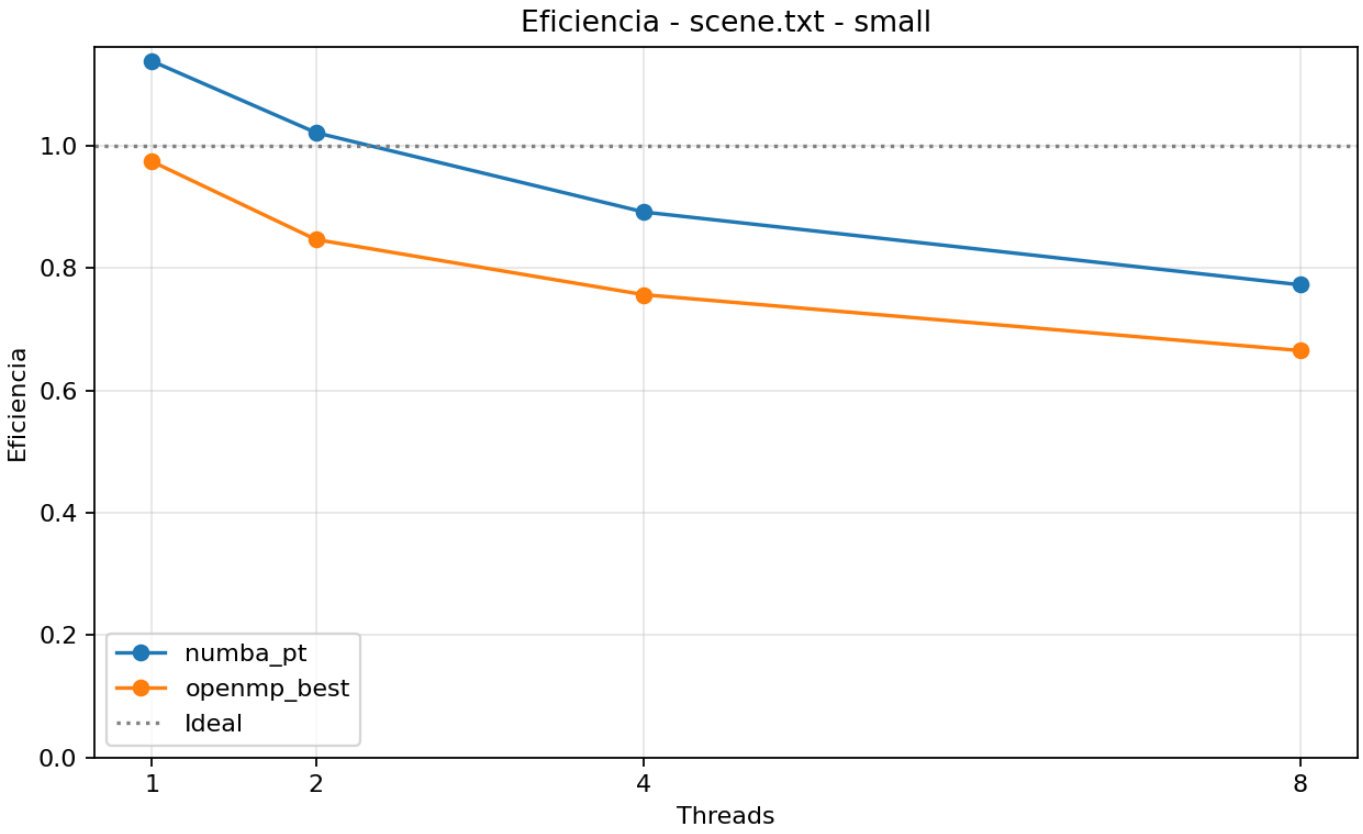

Figure 5: local: Eficiencia scene.txt small

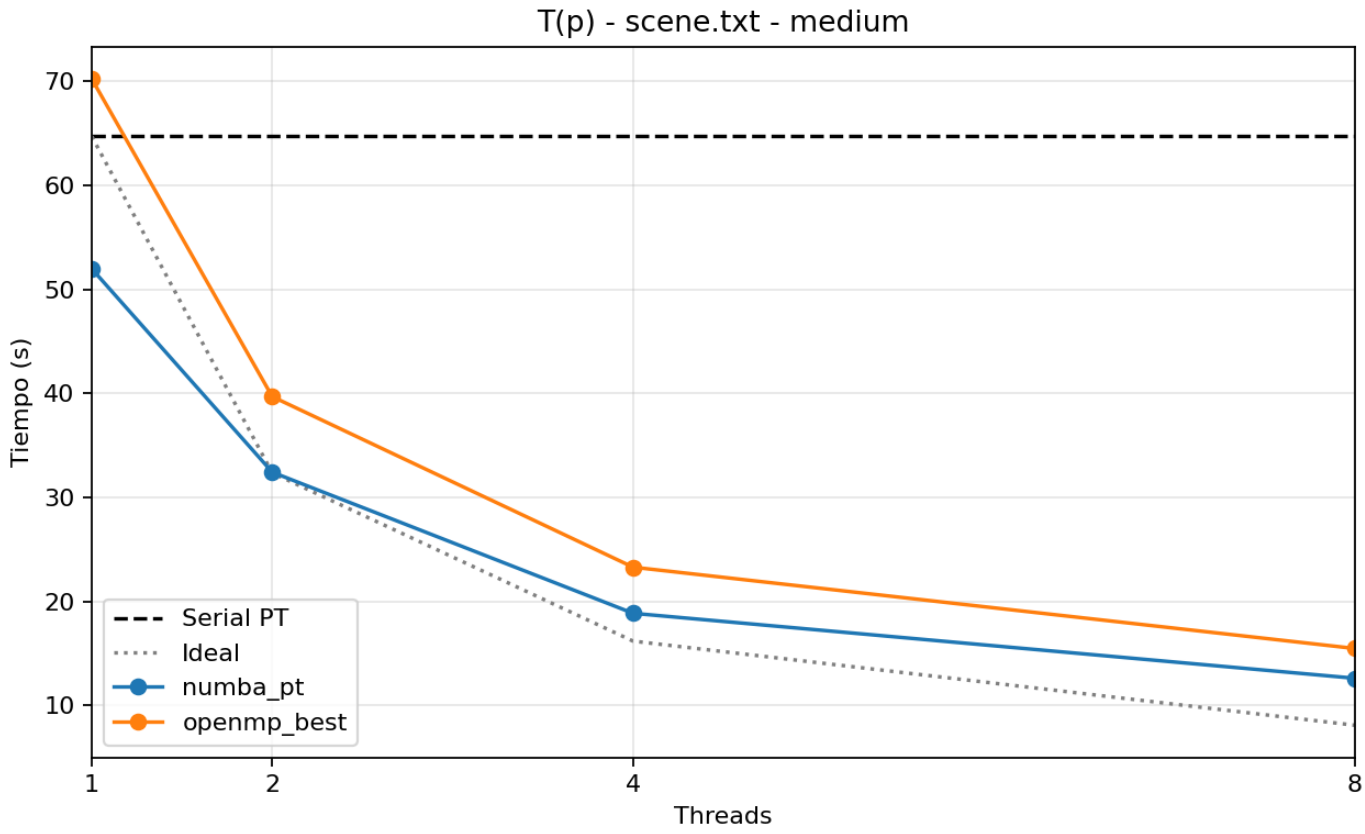

Figure 6: local: Tiempo scene.txt medium

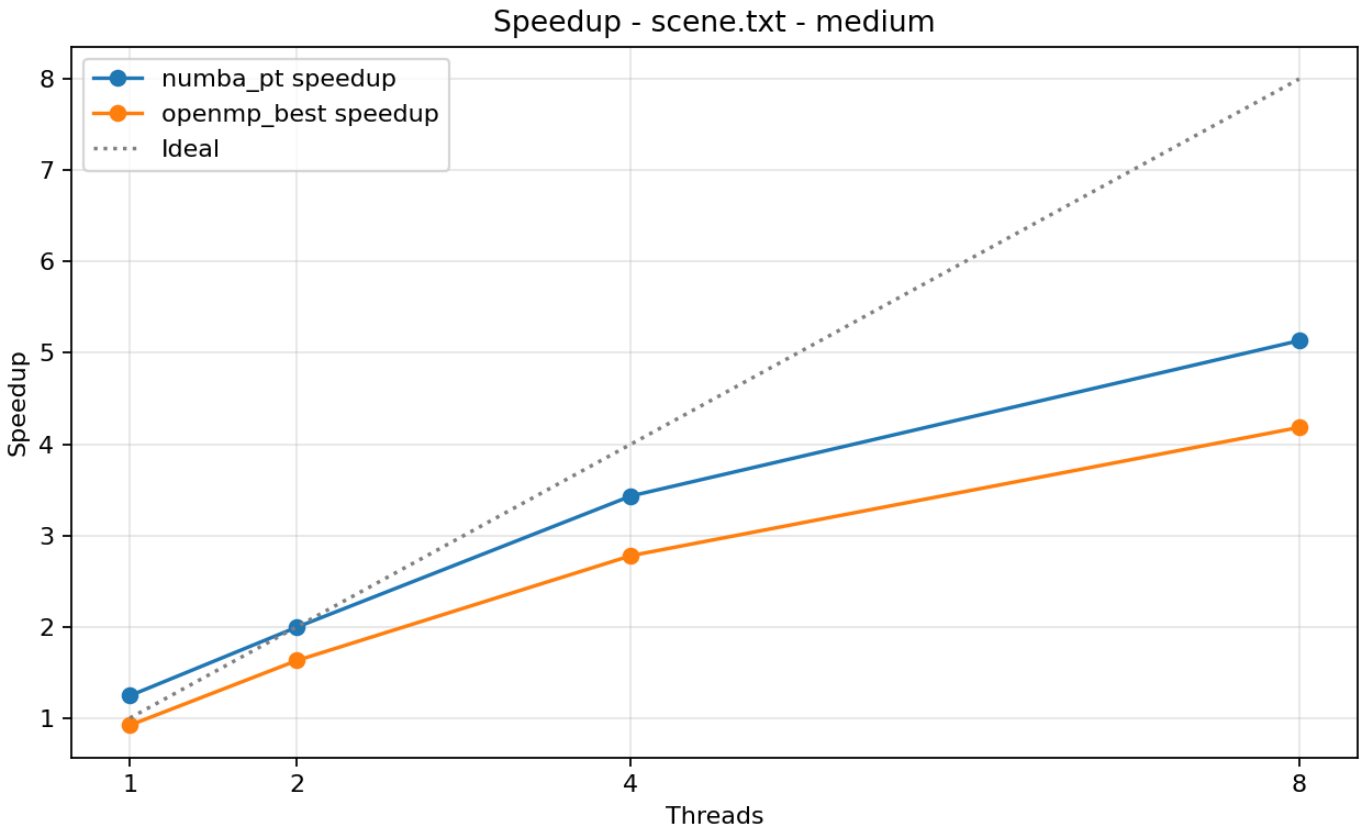

Figure 7: local: Speedup scene.txt medium

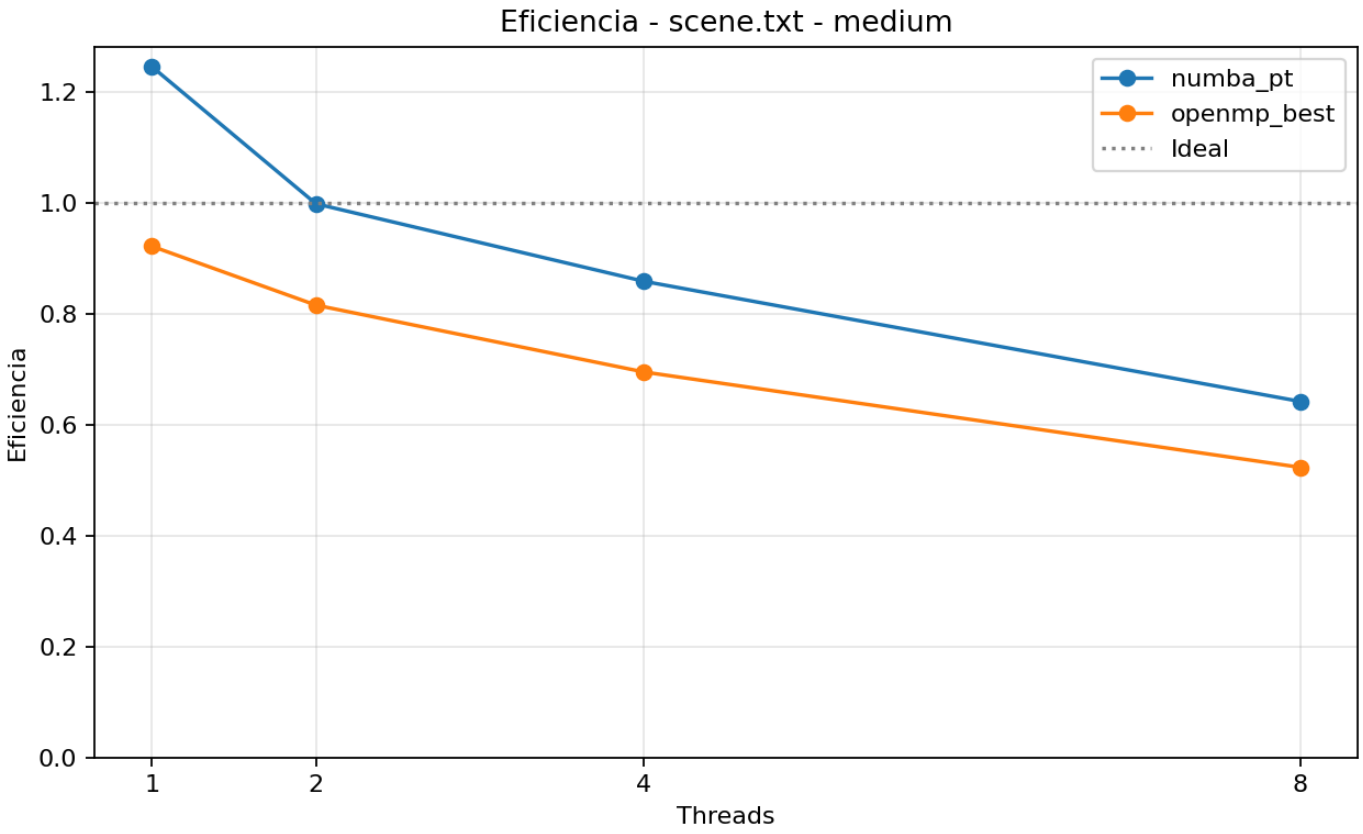

Figure 8: local: Eficiencia scene.txt medium

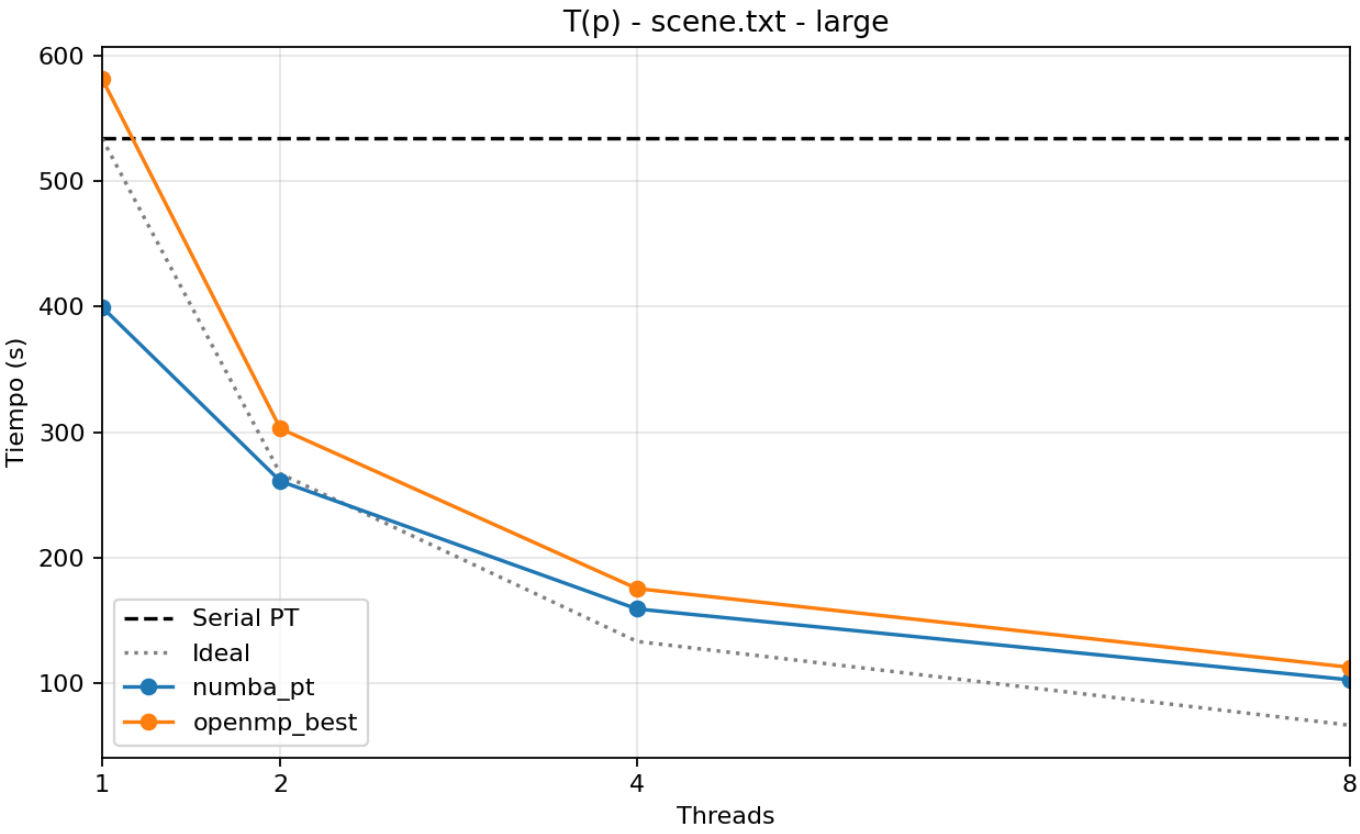

Figure 9: local: Tiempo scene.txt large

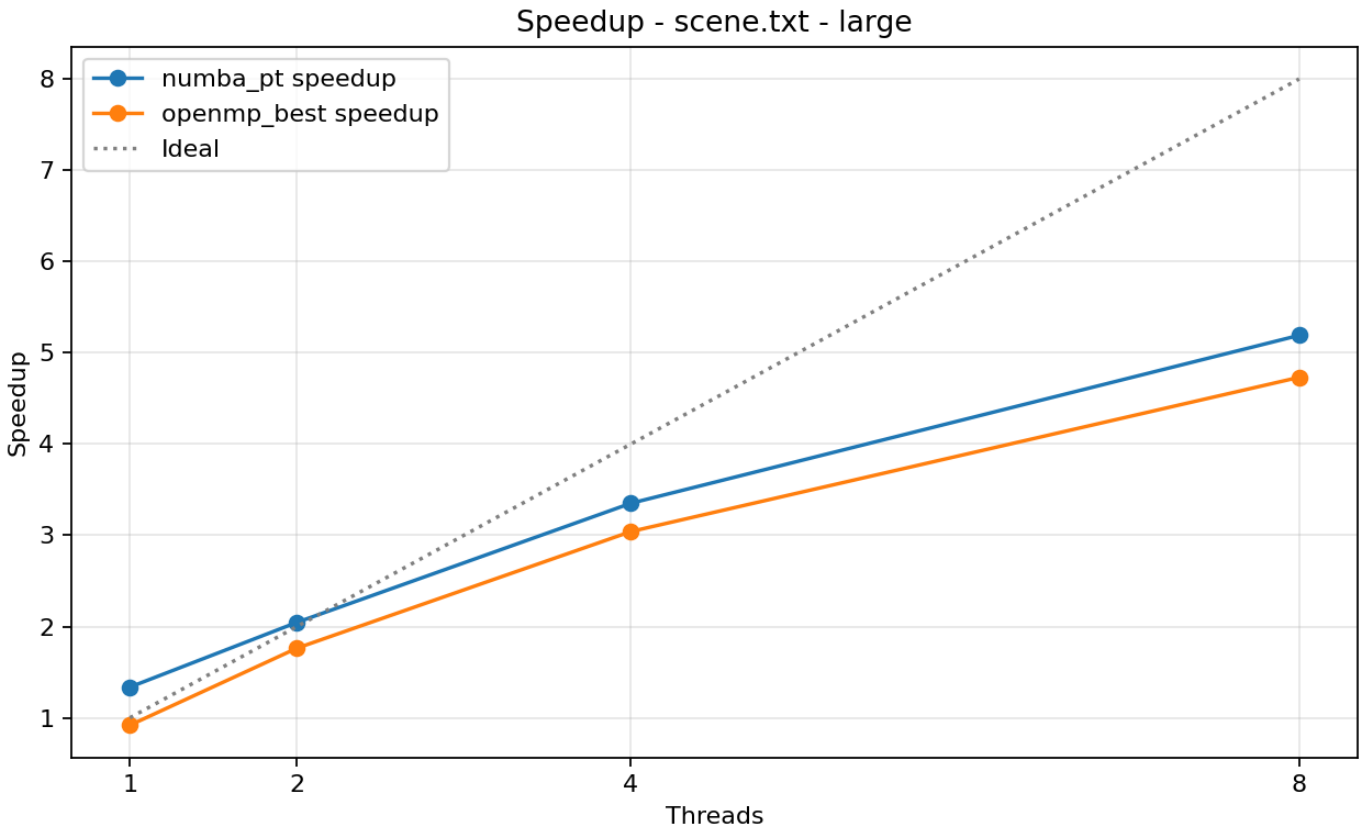

Figure 10: local: Speedup scene.txt large

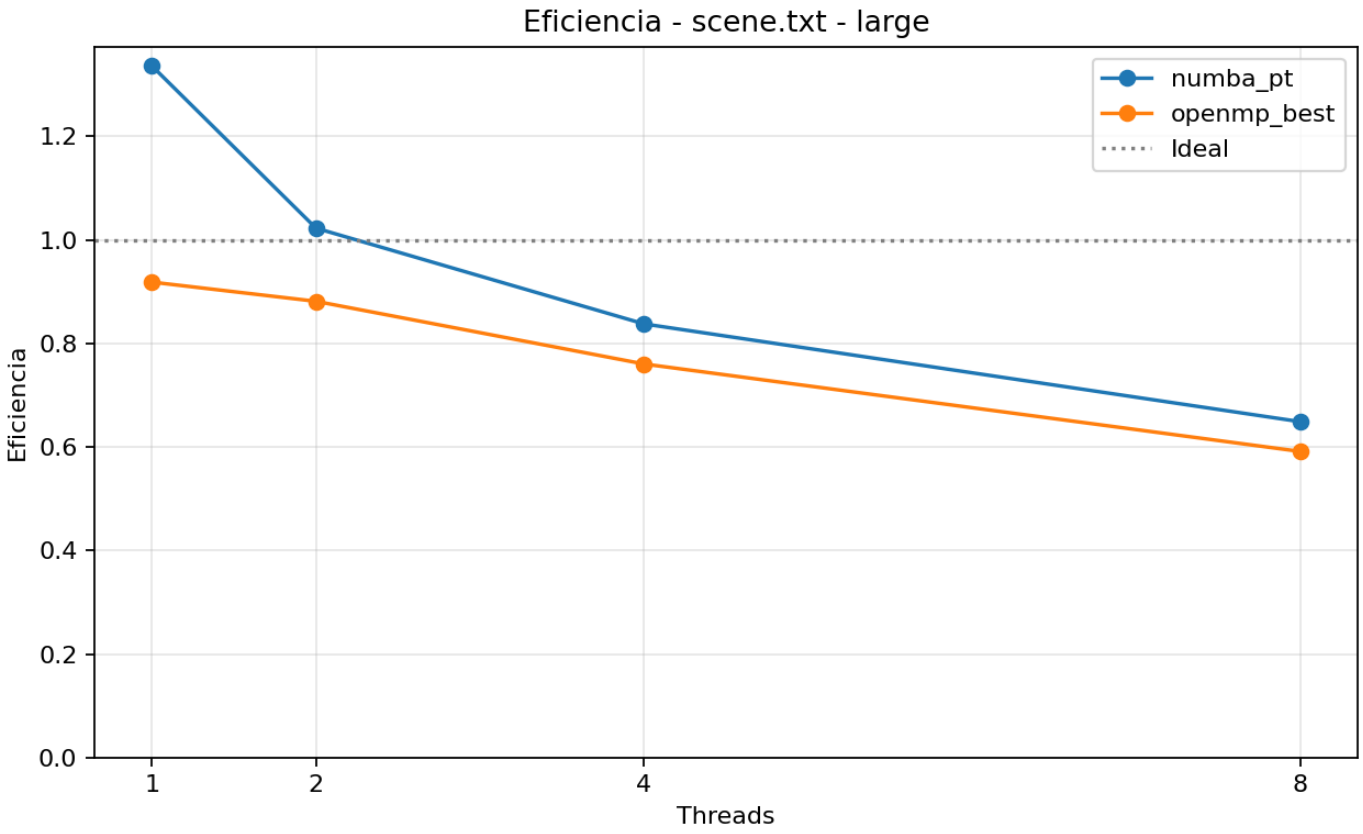

Figure 11: local: Eficiencia scene.txt large

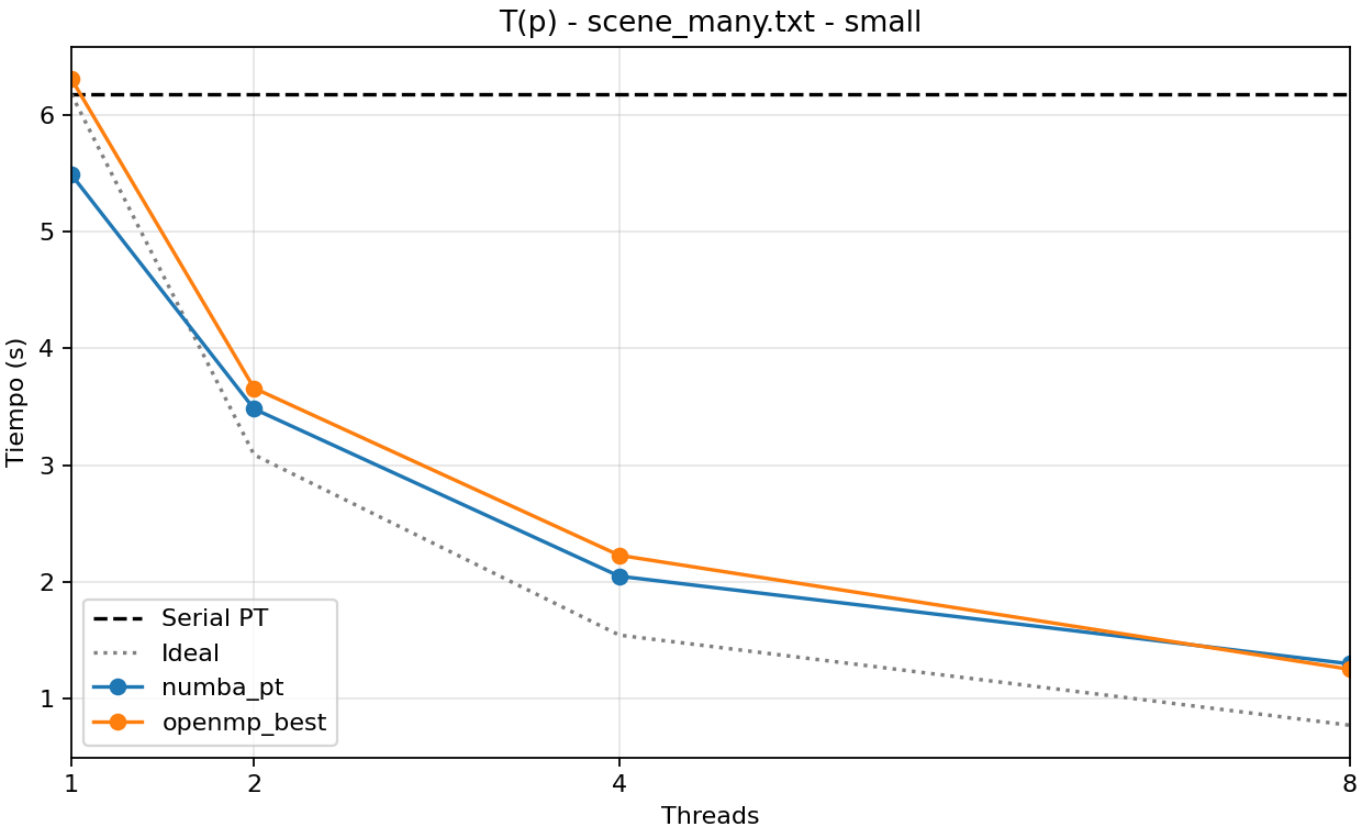

Figure 12: local: Tiempo scene_many.txt small

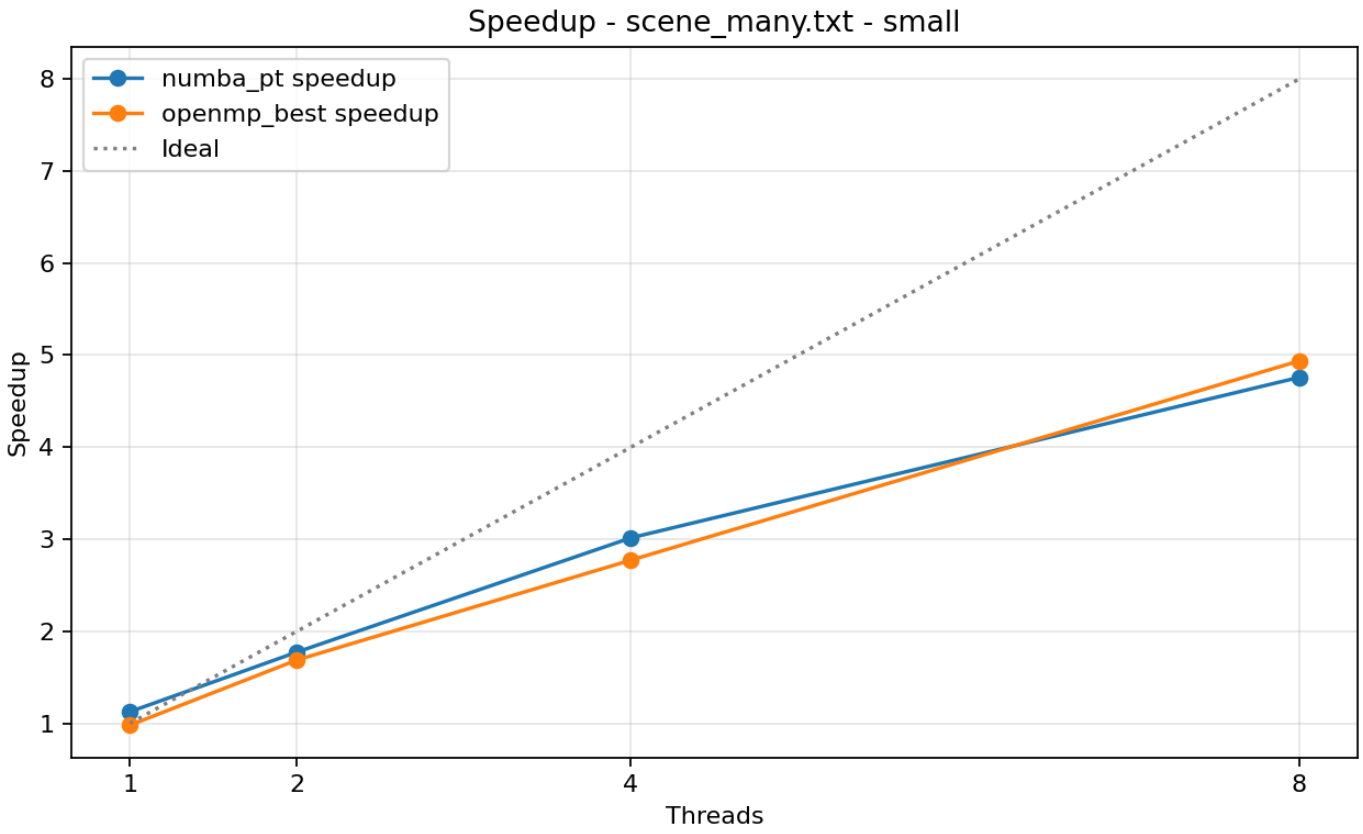

Figure 13: local: Speedup scene_many.txt small

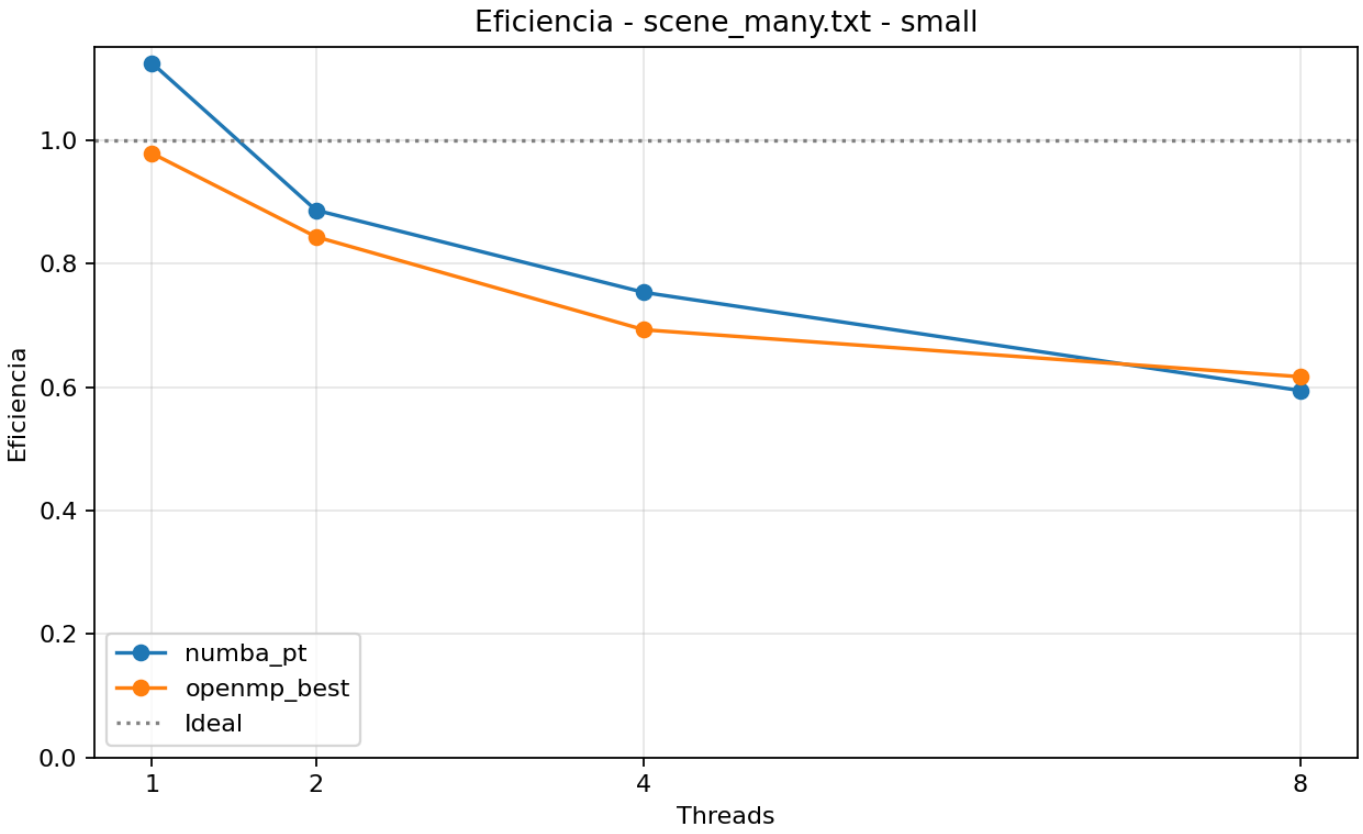

Figure 14: local: Eficiencia scene_many.txt small

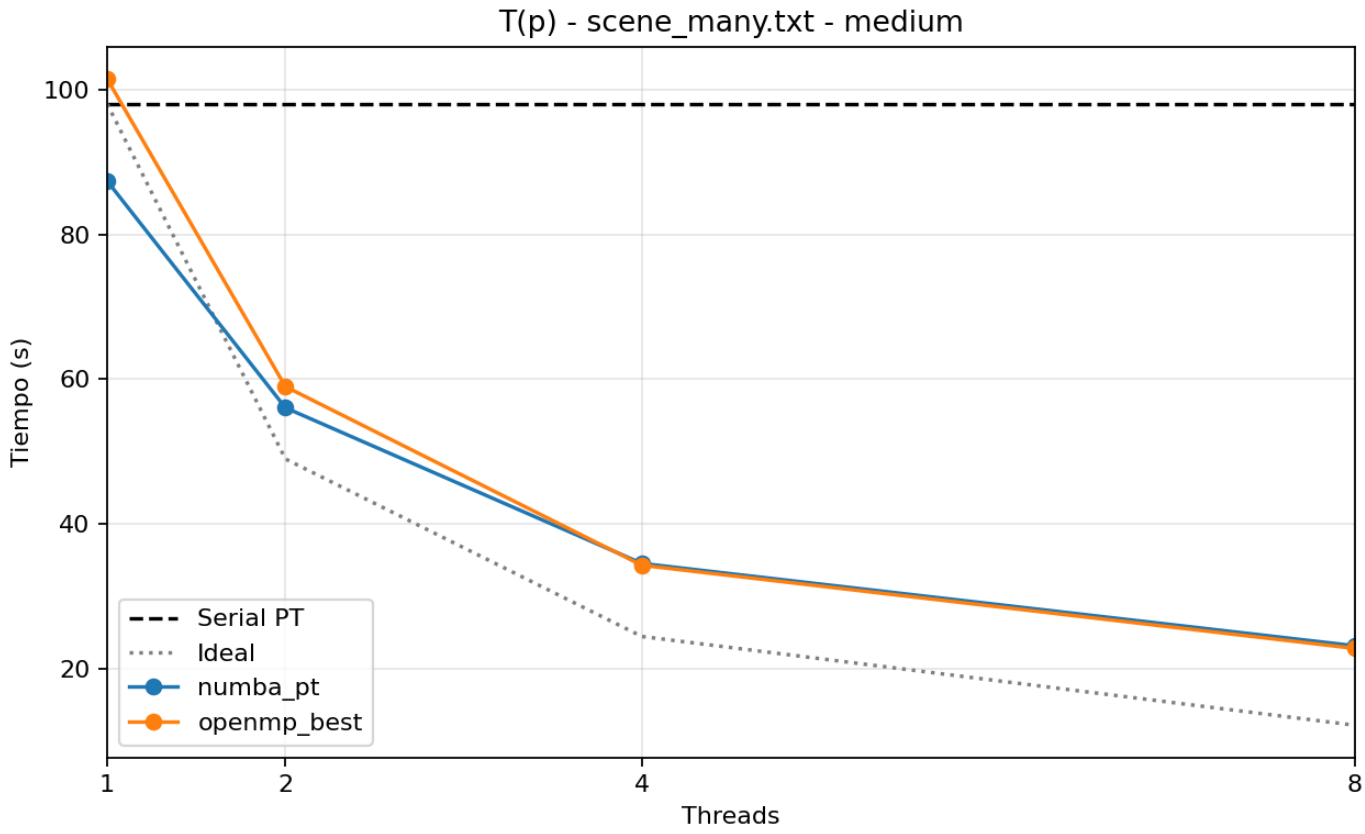

Figure 15: local: Tiempo scene_many.txt medium

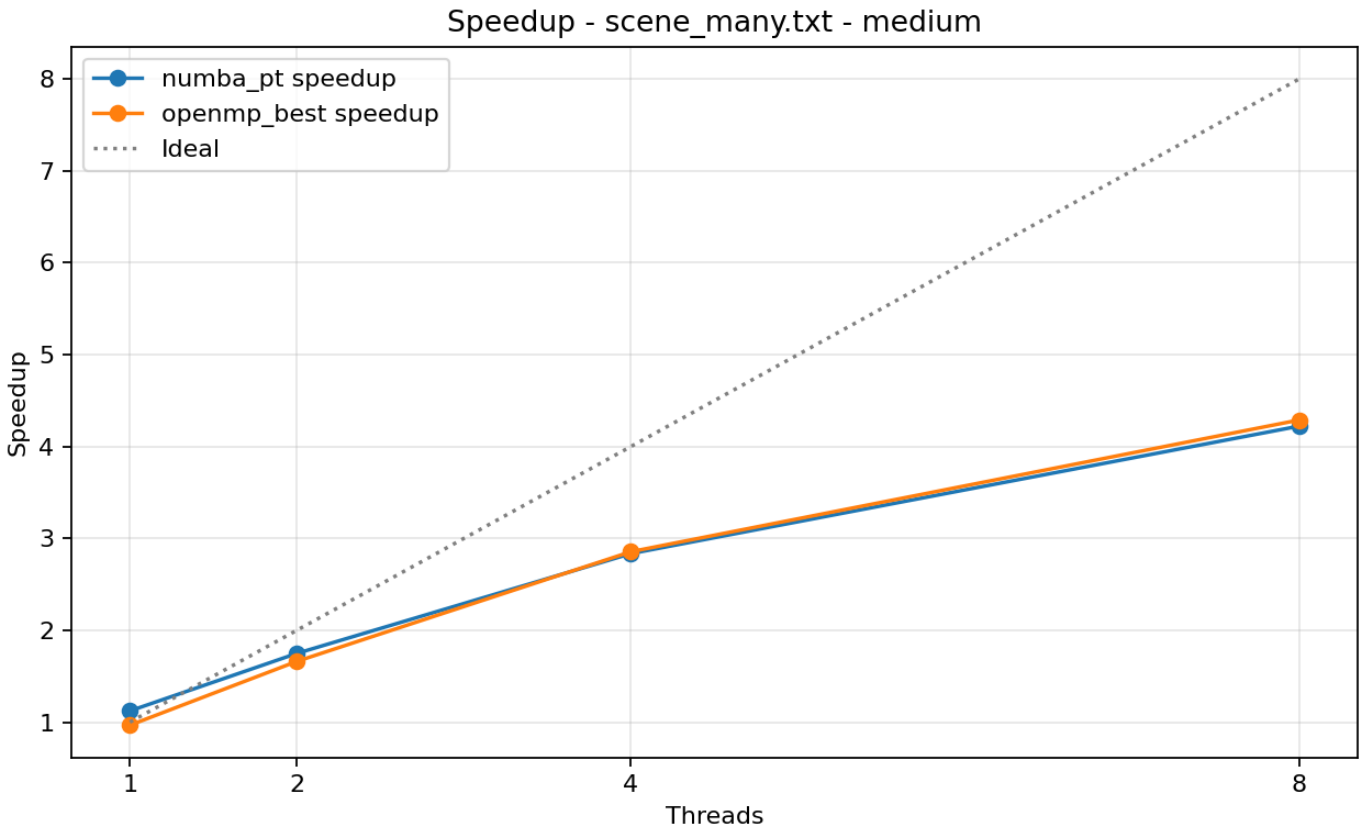

Figure 16: local: Speedup scene_many.txt medium

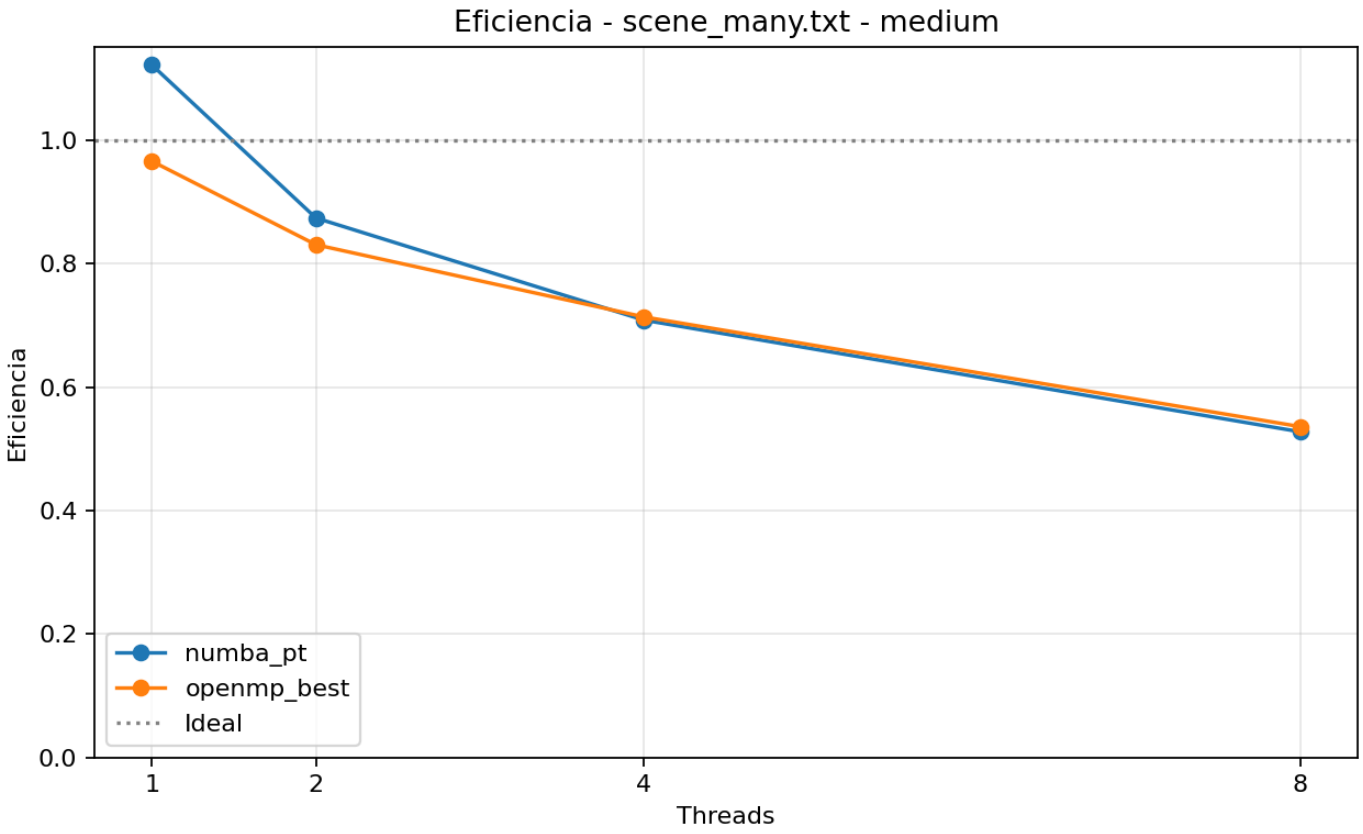

Figure 17: local: Eficiencia scene_many.txt medium

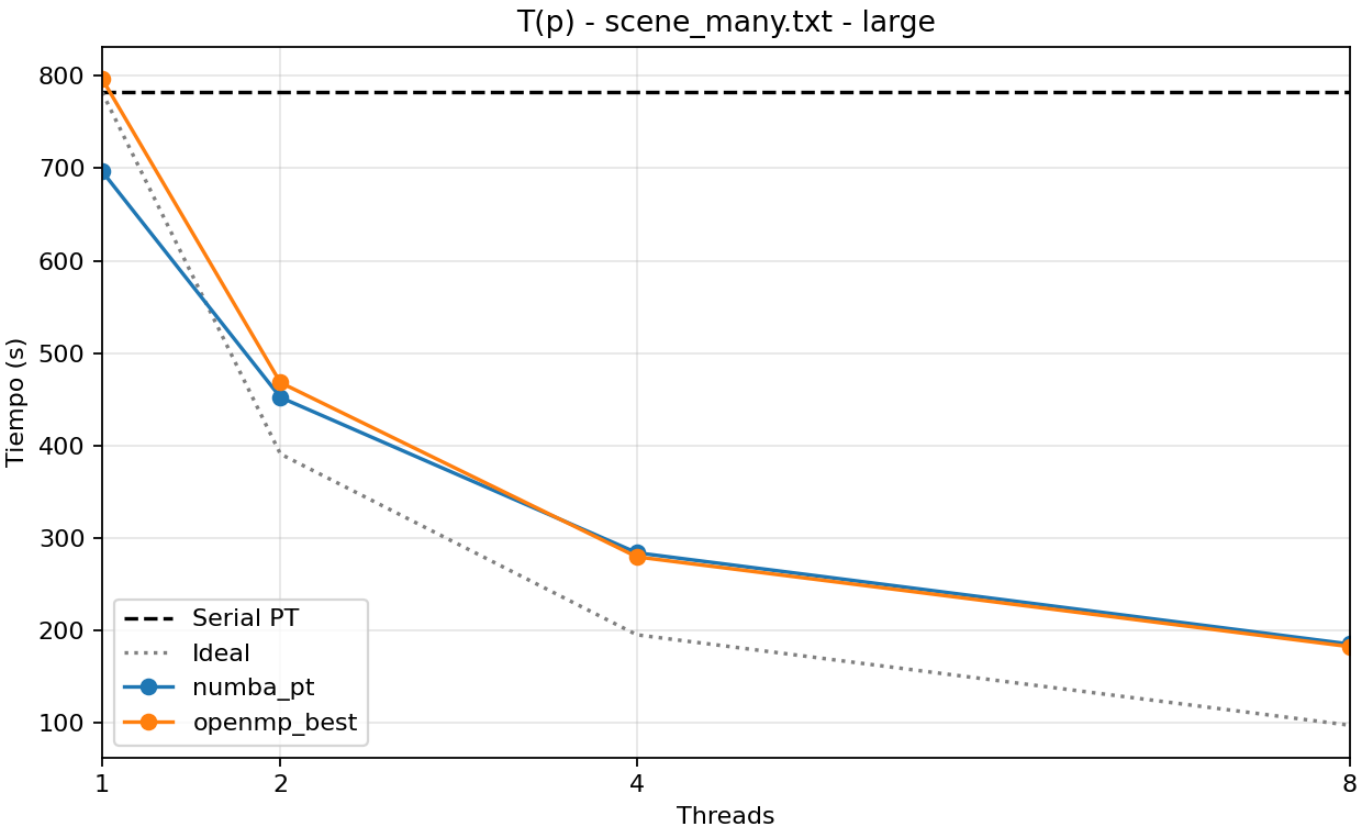

Figure 18: local: Tiempo scene_many.txt large

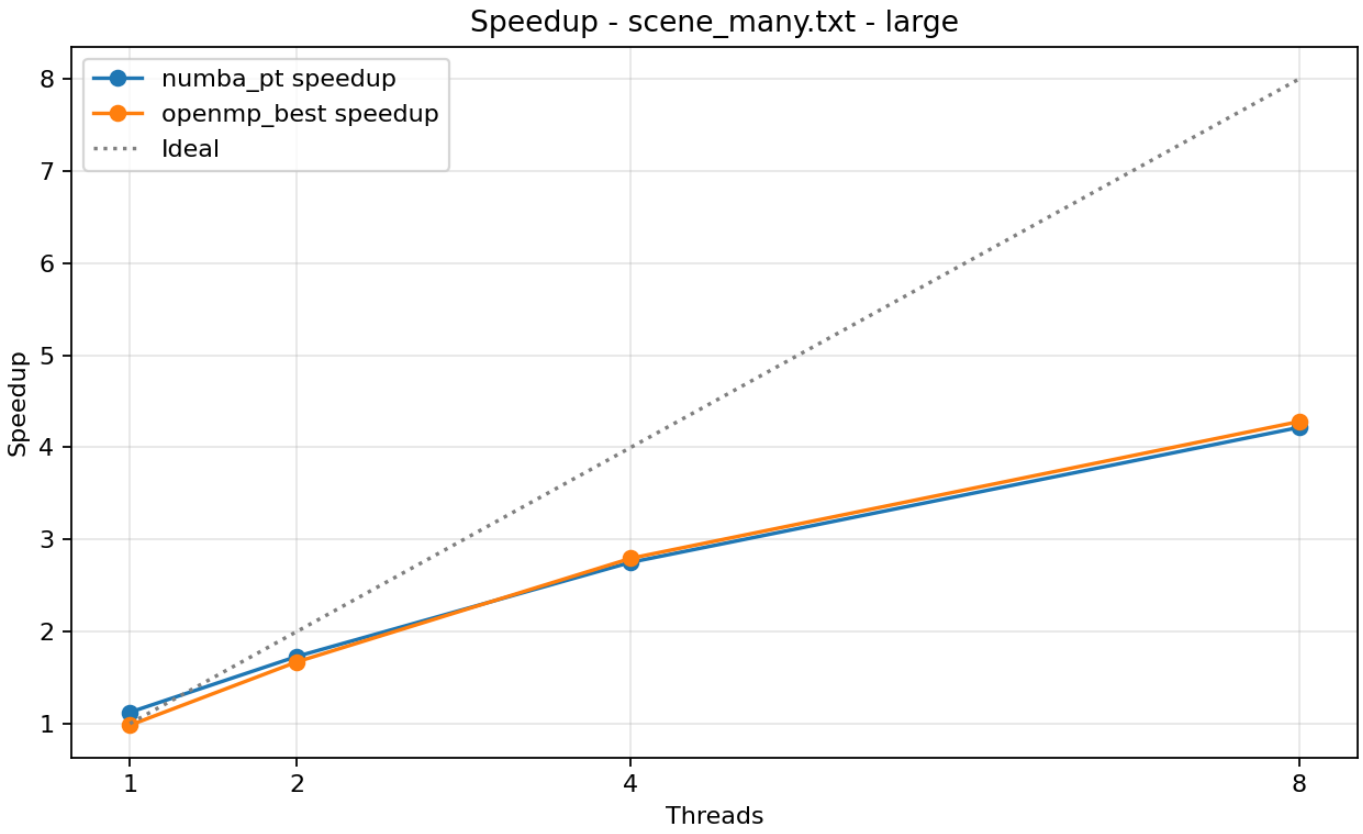

Figure 19: local: Speedup scene_many.txt large

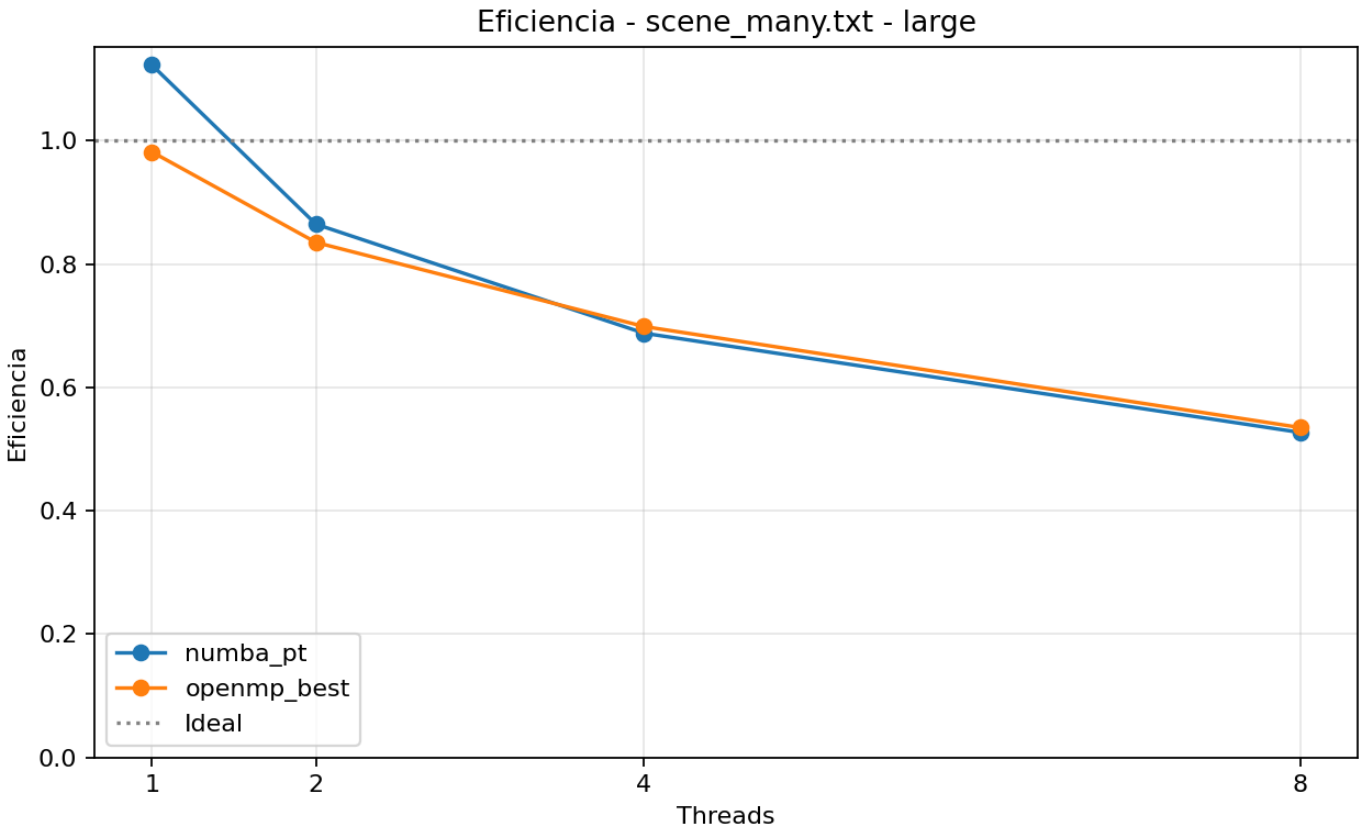

Figure 20: local: Eficiencia scene_many.txt large

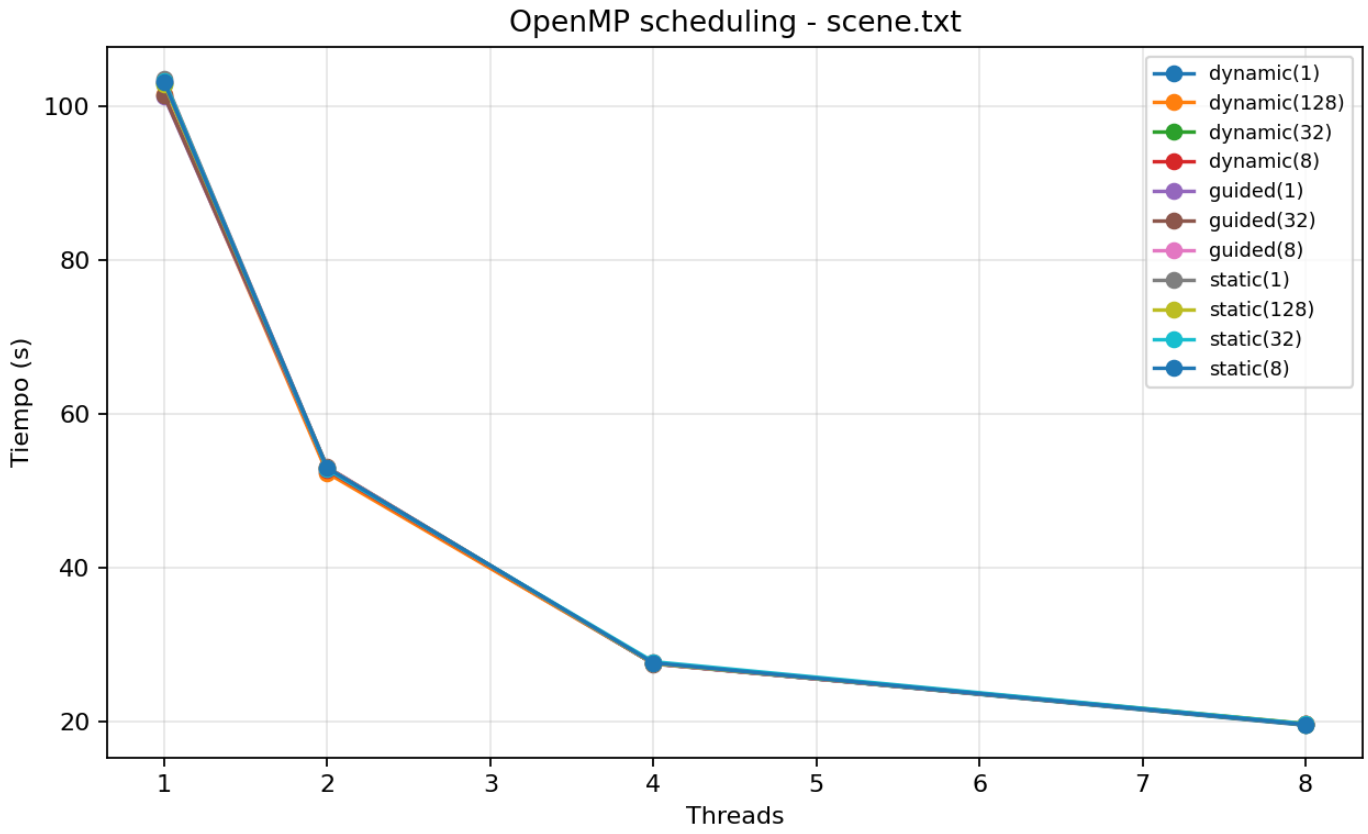

Figure 21: cluster: Scheduling scene.txt

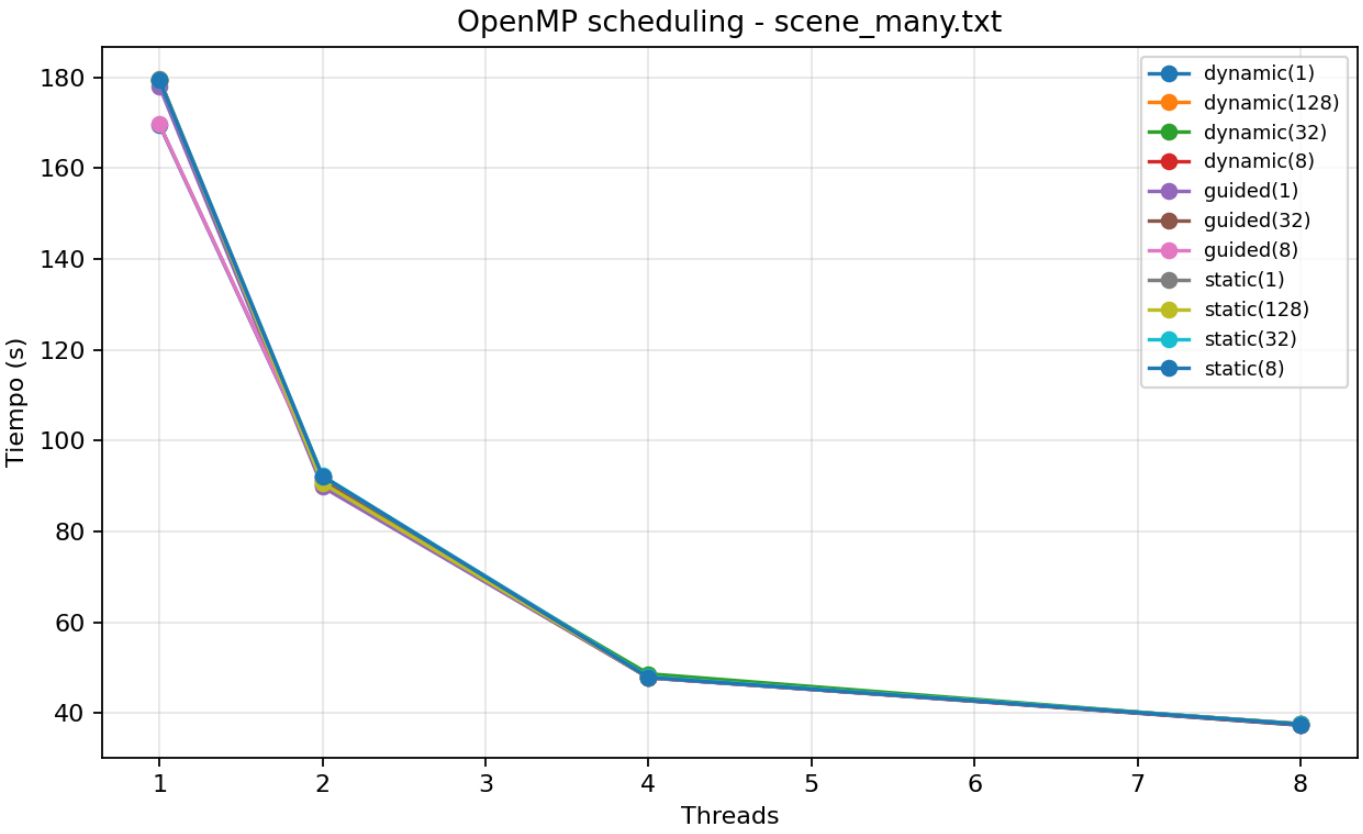

Figure 22: cluster: Scheduling scene_many.txt

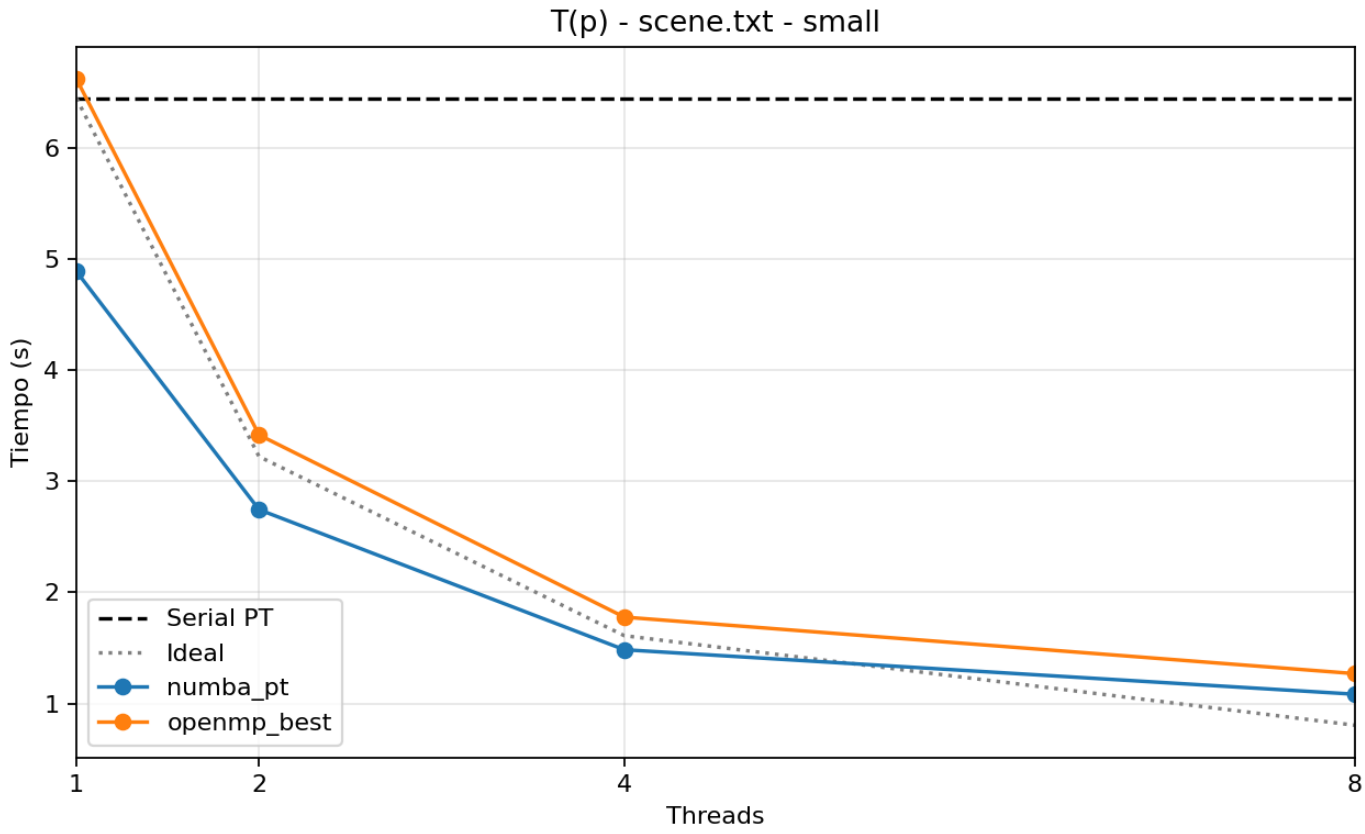

Figure 23: cluster: Tiempo scene.txt small

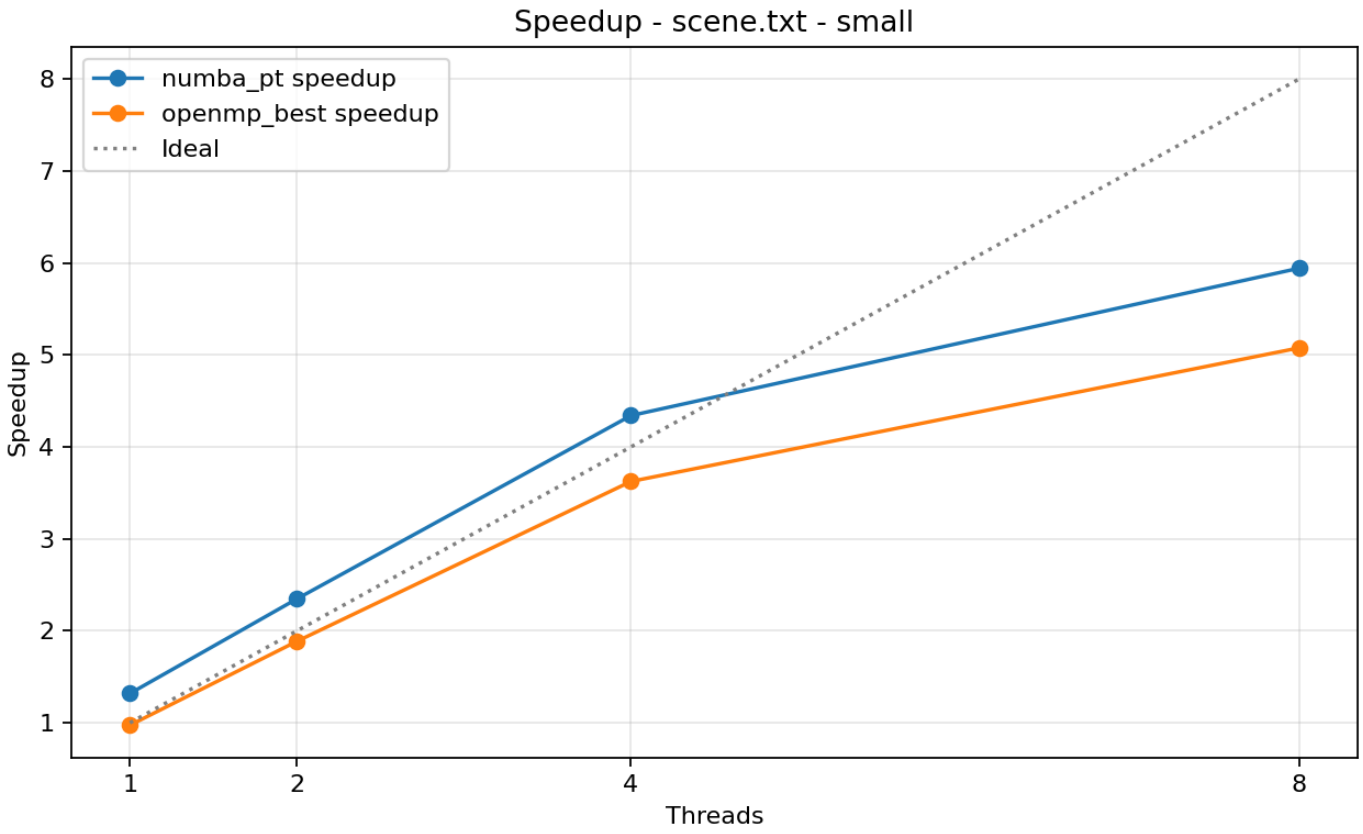

Figure 24: cluster: Speedup scene.txt small

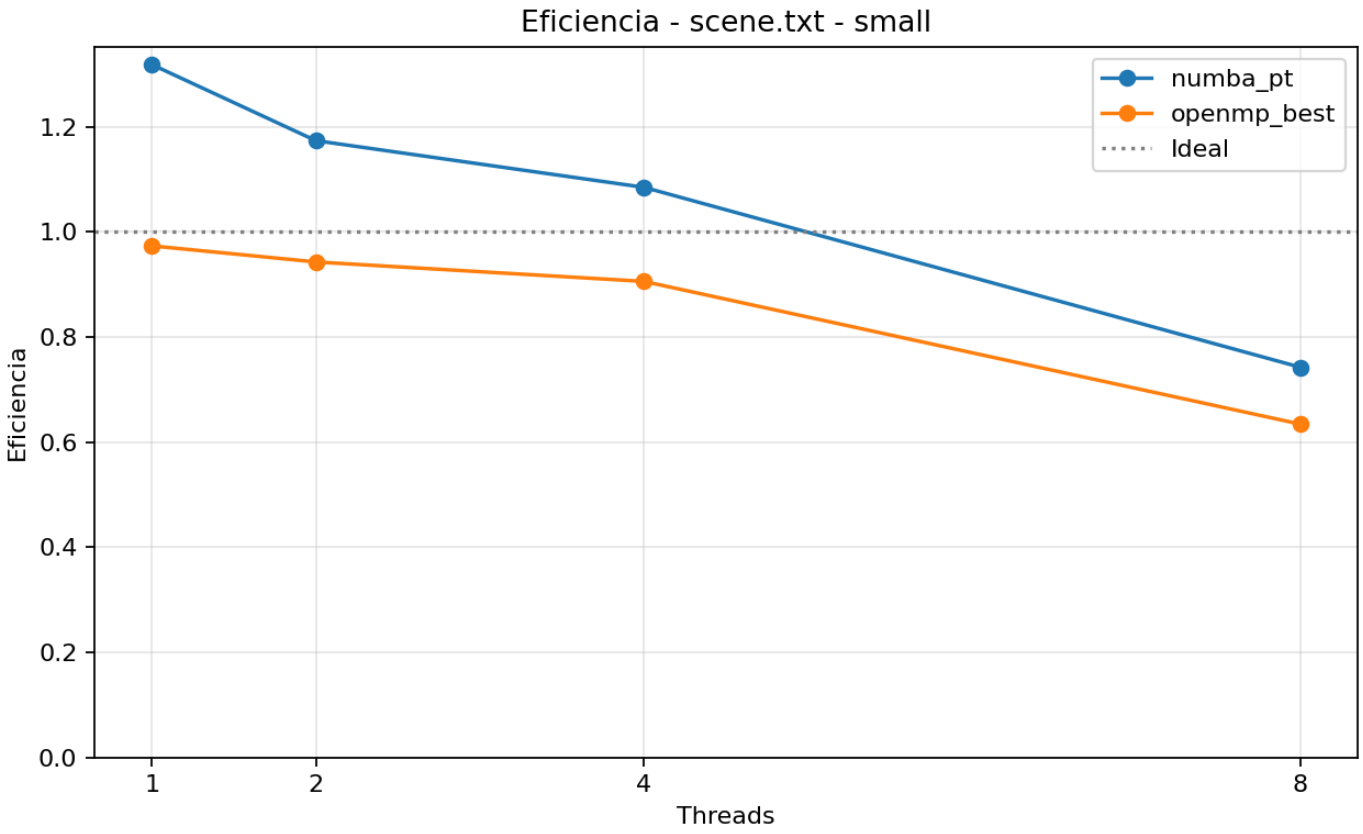

Figure 25: cluster: Eficiencia scene.txt small

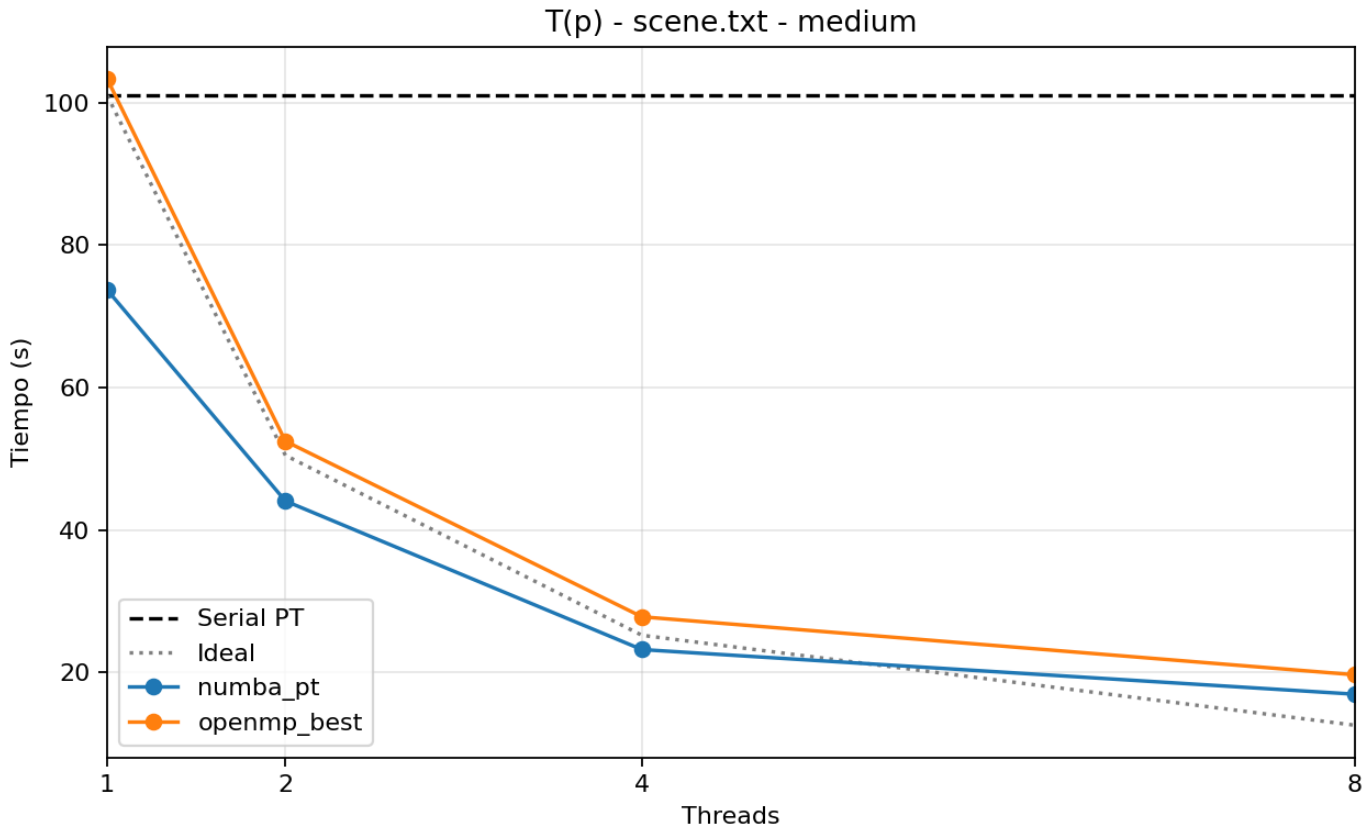

Figure 26: cluster: Tiempo scene.txt medium

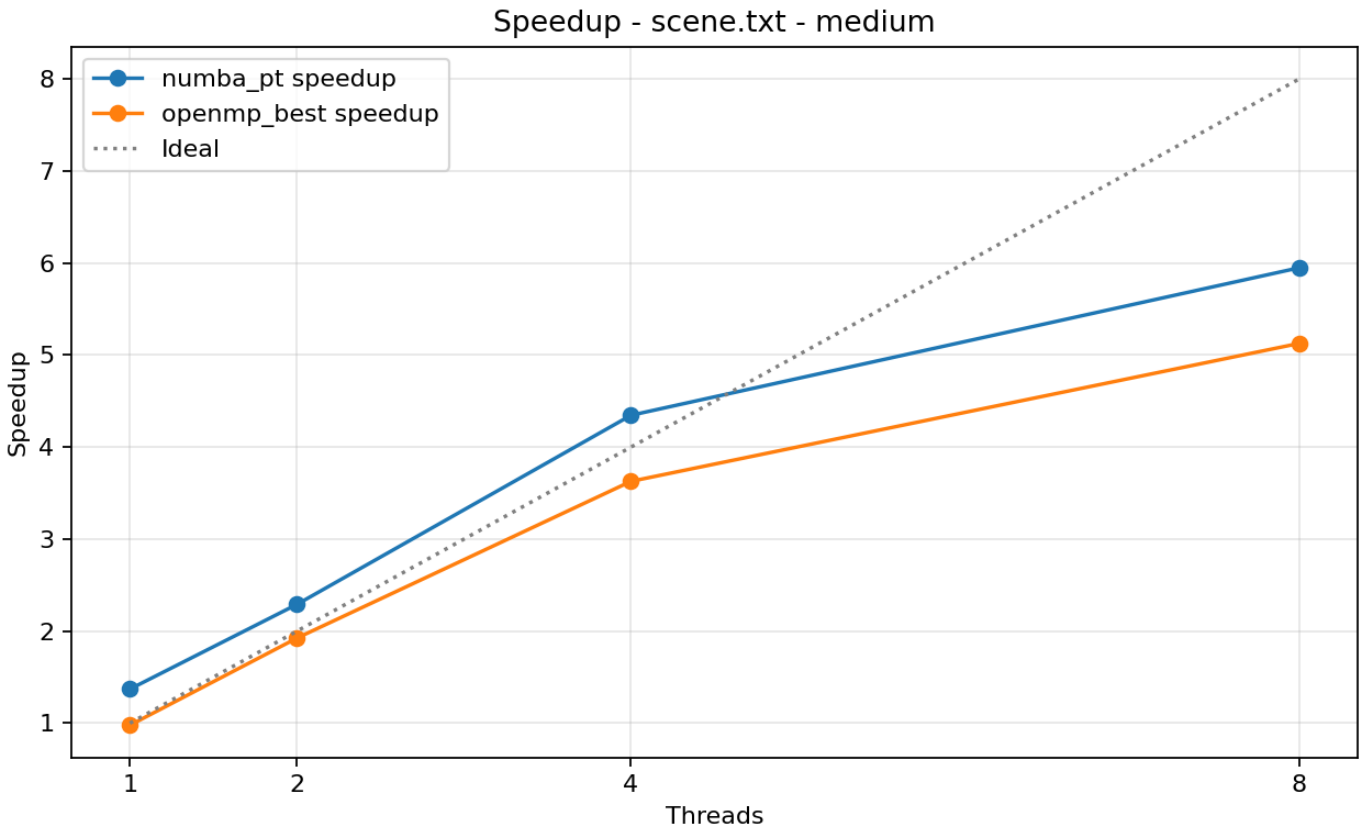

Figure 27: cluster: Speedup scene.txt medium

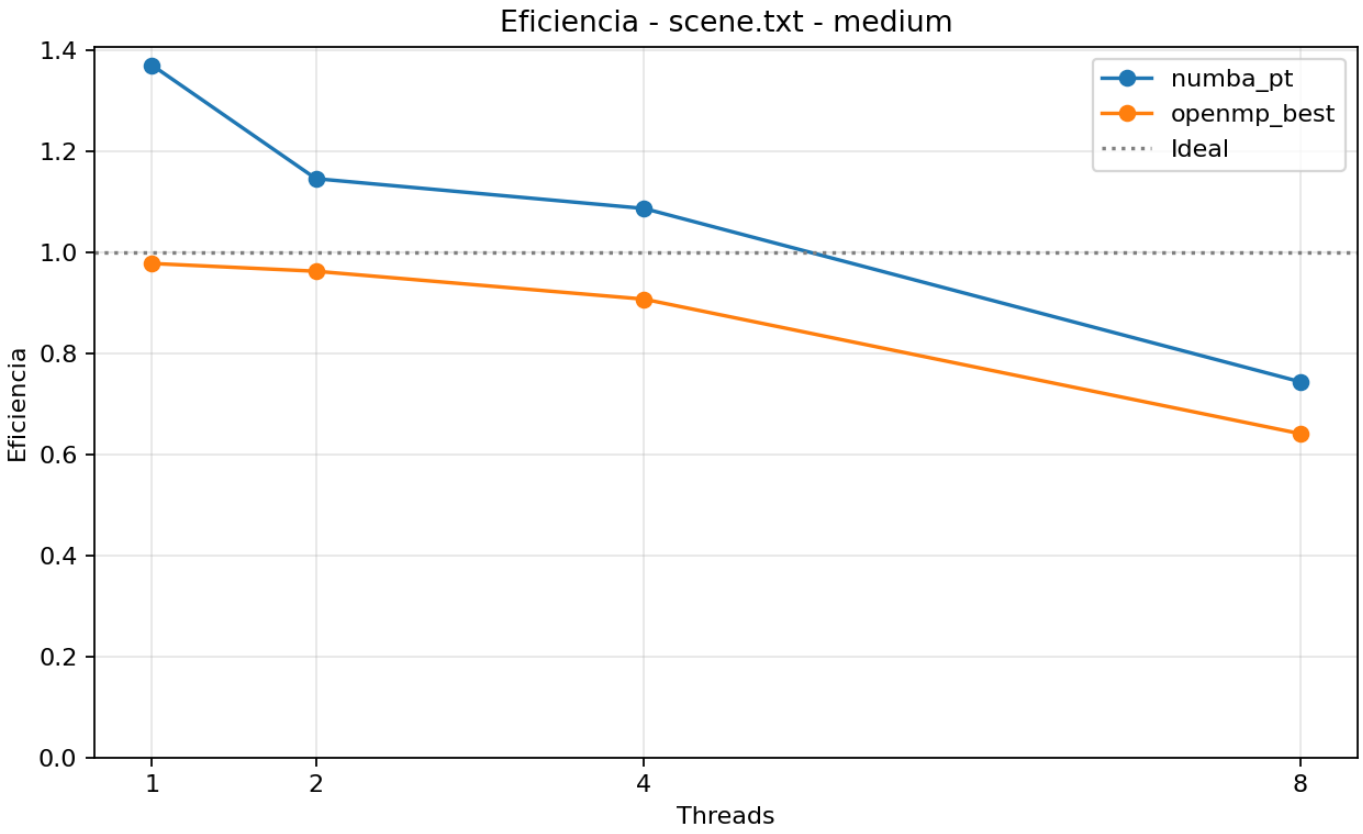

Figure 28: cluster: Eficiencia scene.txt medium

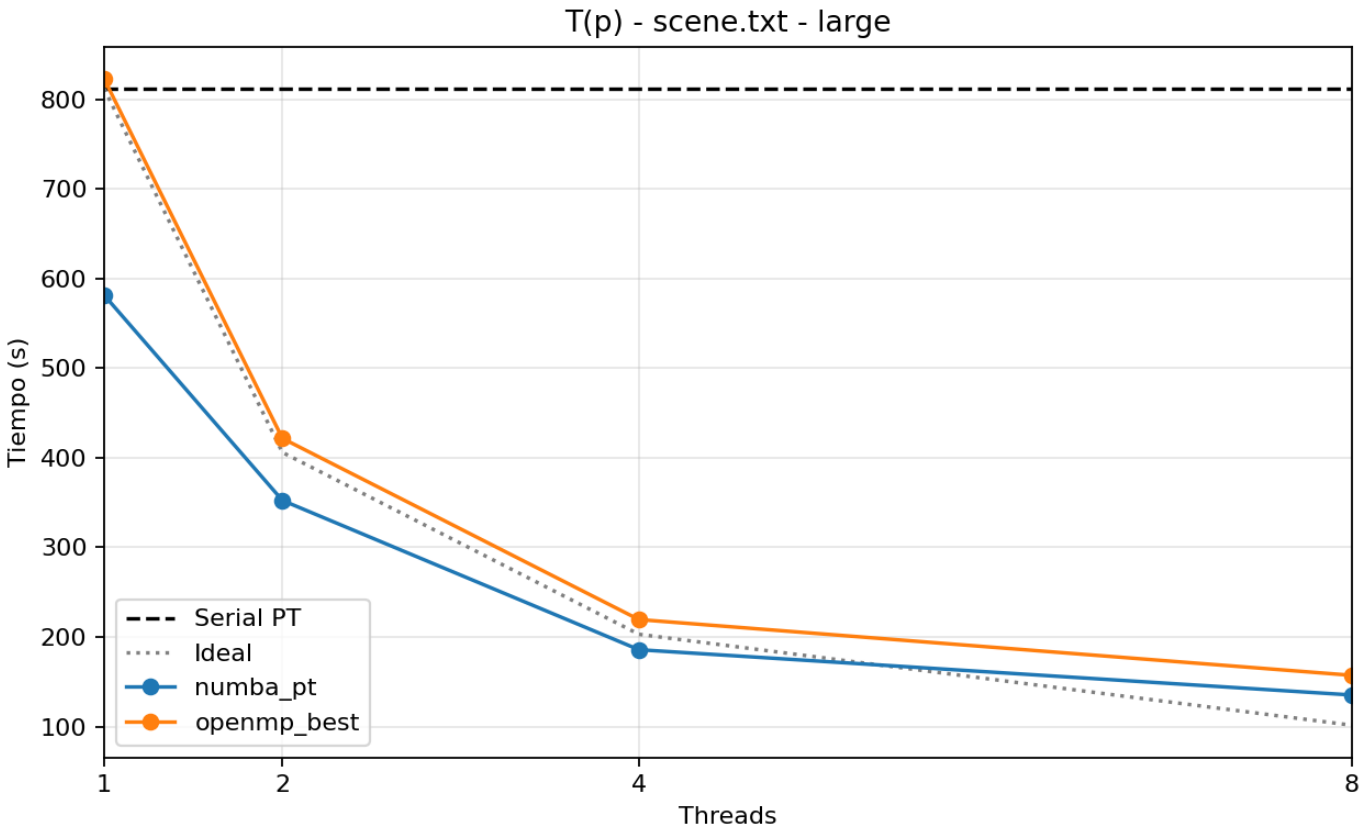

Figure 29: cluster: Tiempo scene.txt large

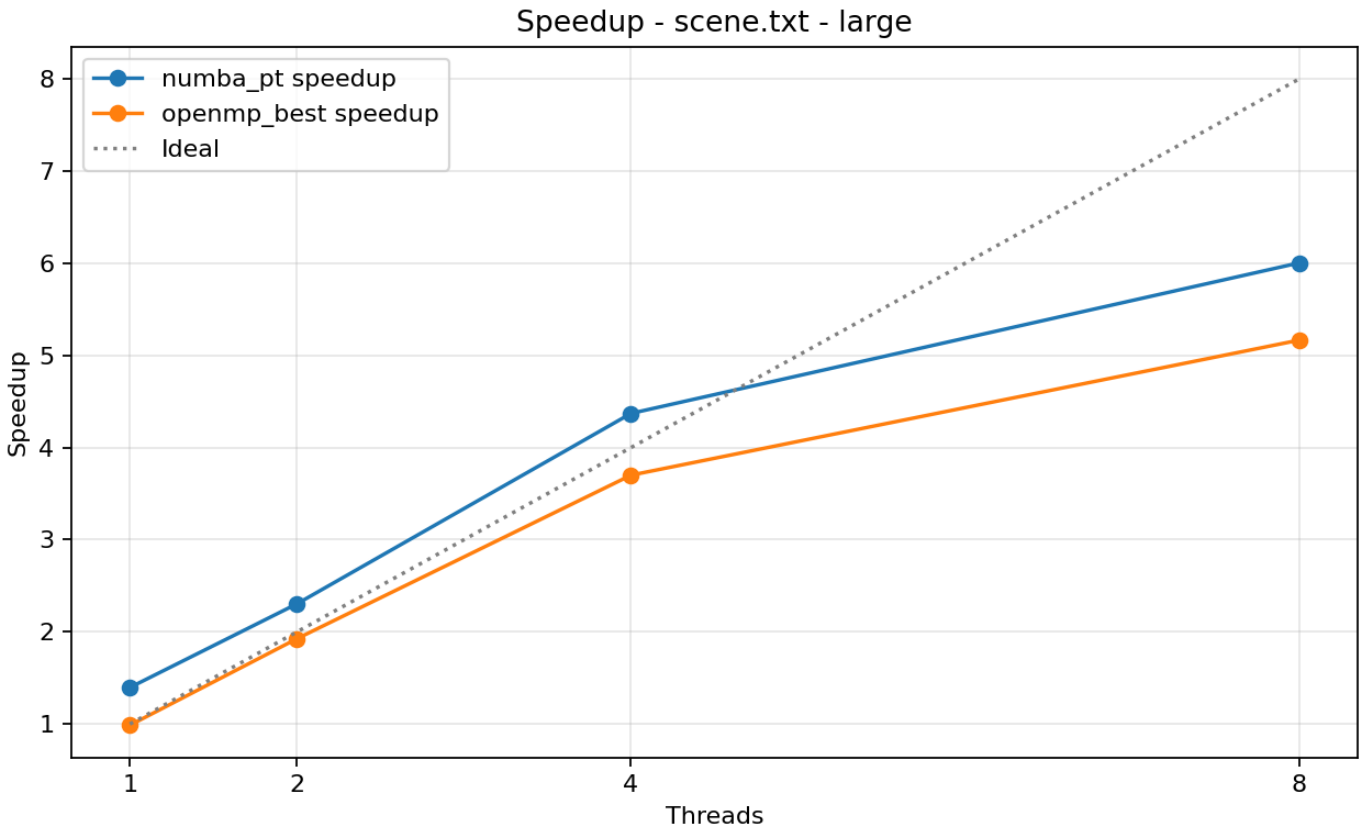

Figure 30: cluster: Speedup scene.txt large

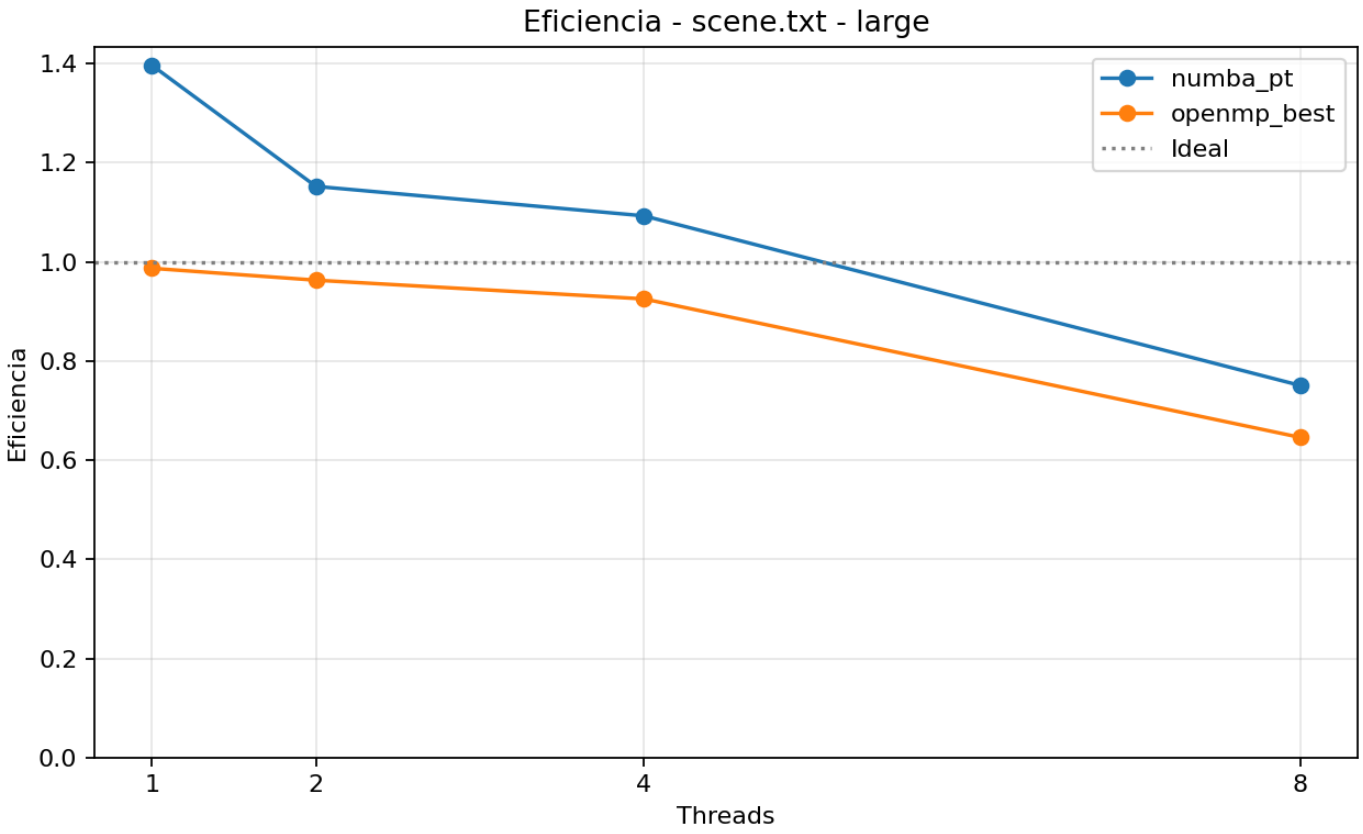

Figure 31: cluster: Eficiencia scene.txt large

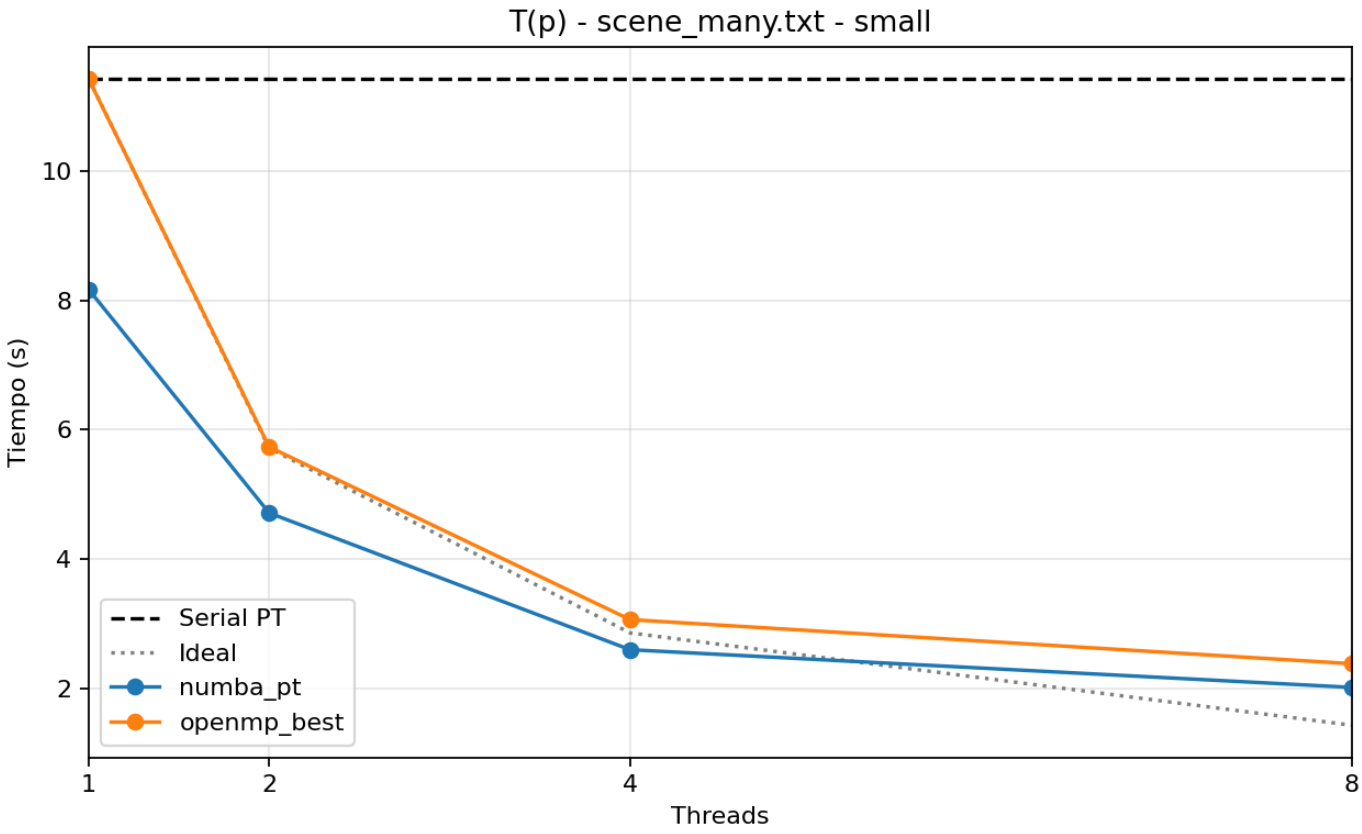

Figure 32: cluster: Tiempo scene_many.txt small

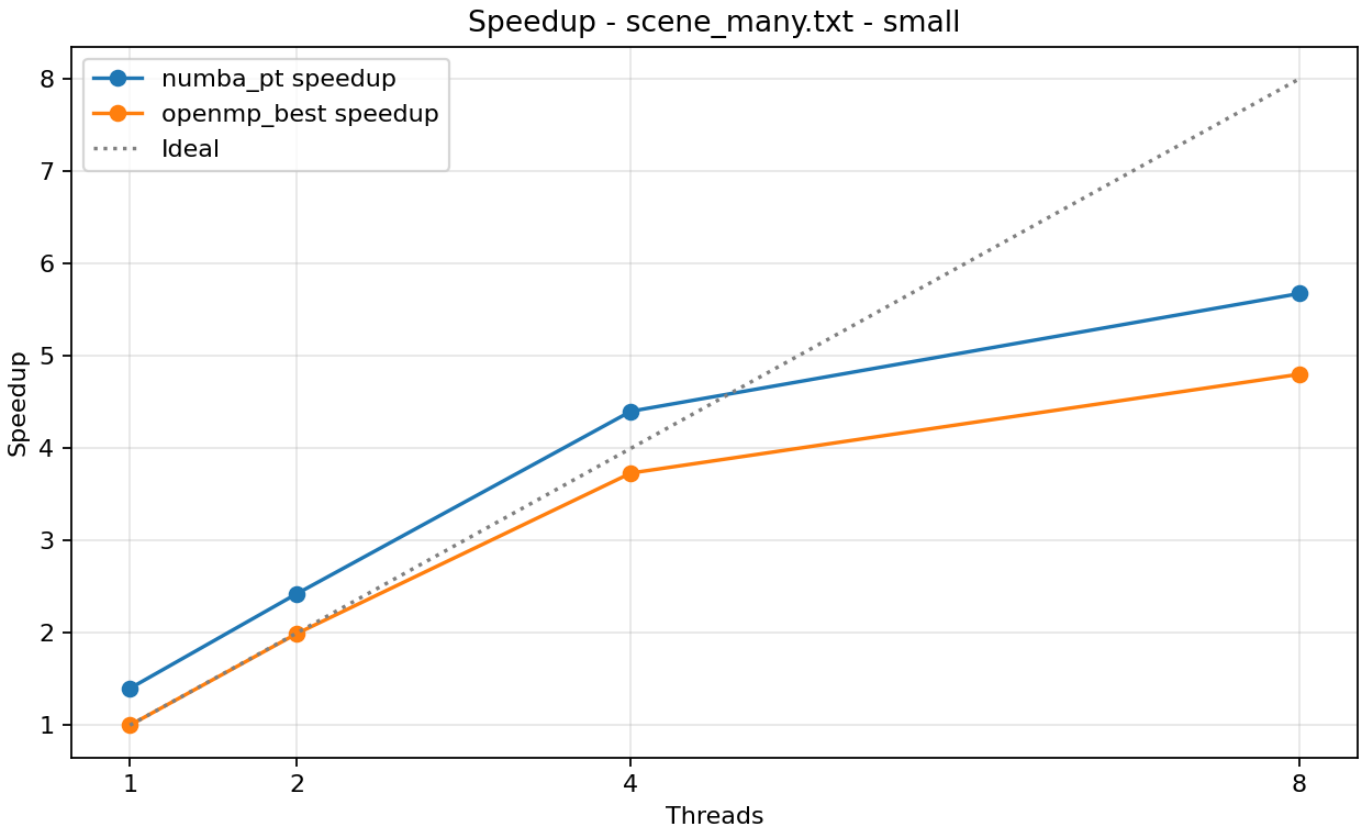

Figure 33: cluster: Speedup scene_many.txt small

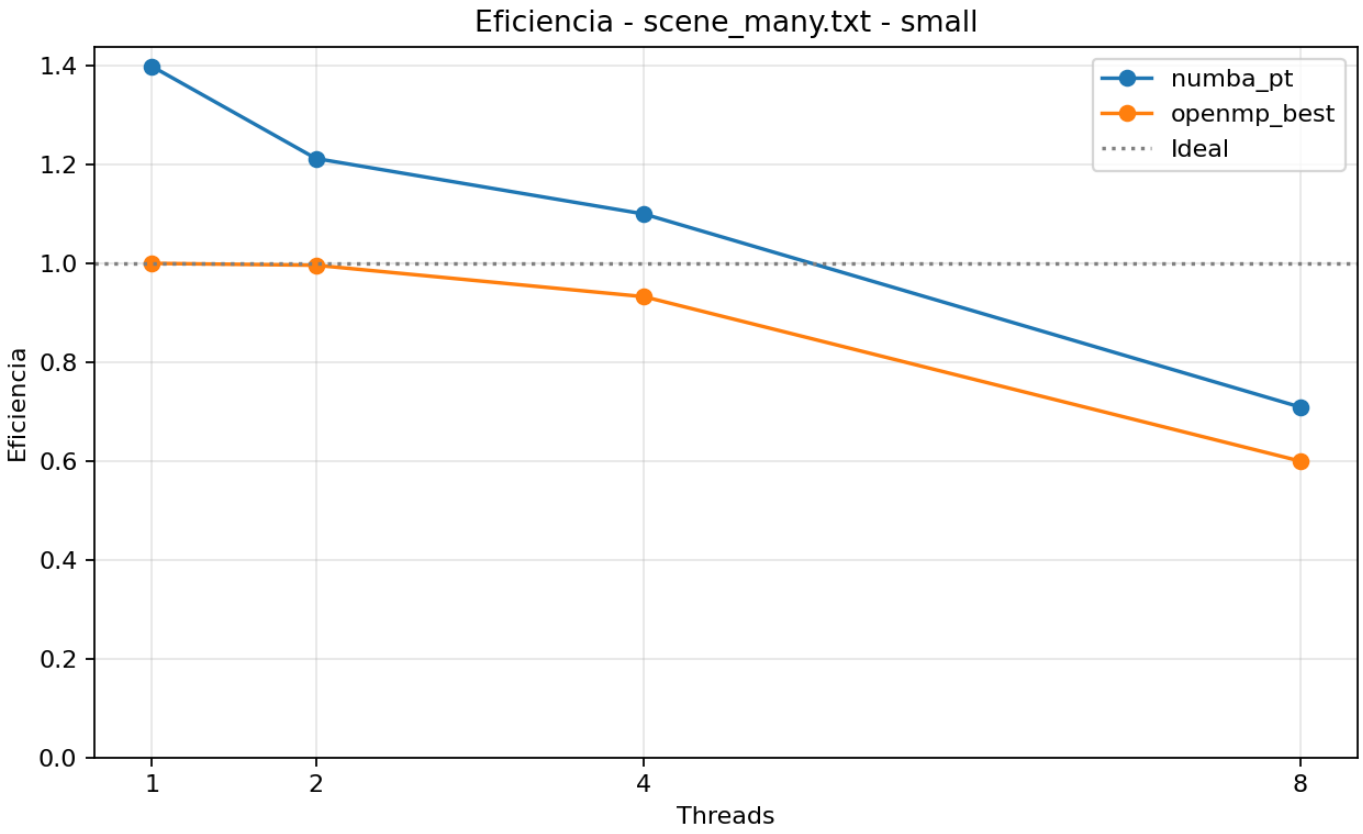

Figure 34: cluster: Eficiencia scene_many.txt small

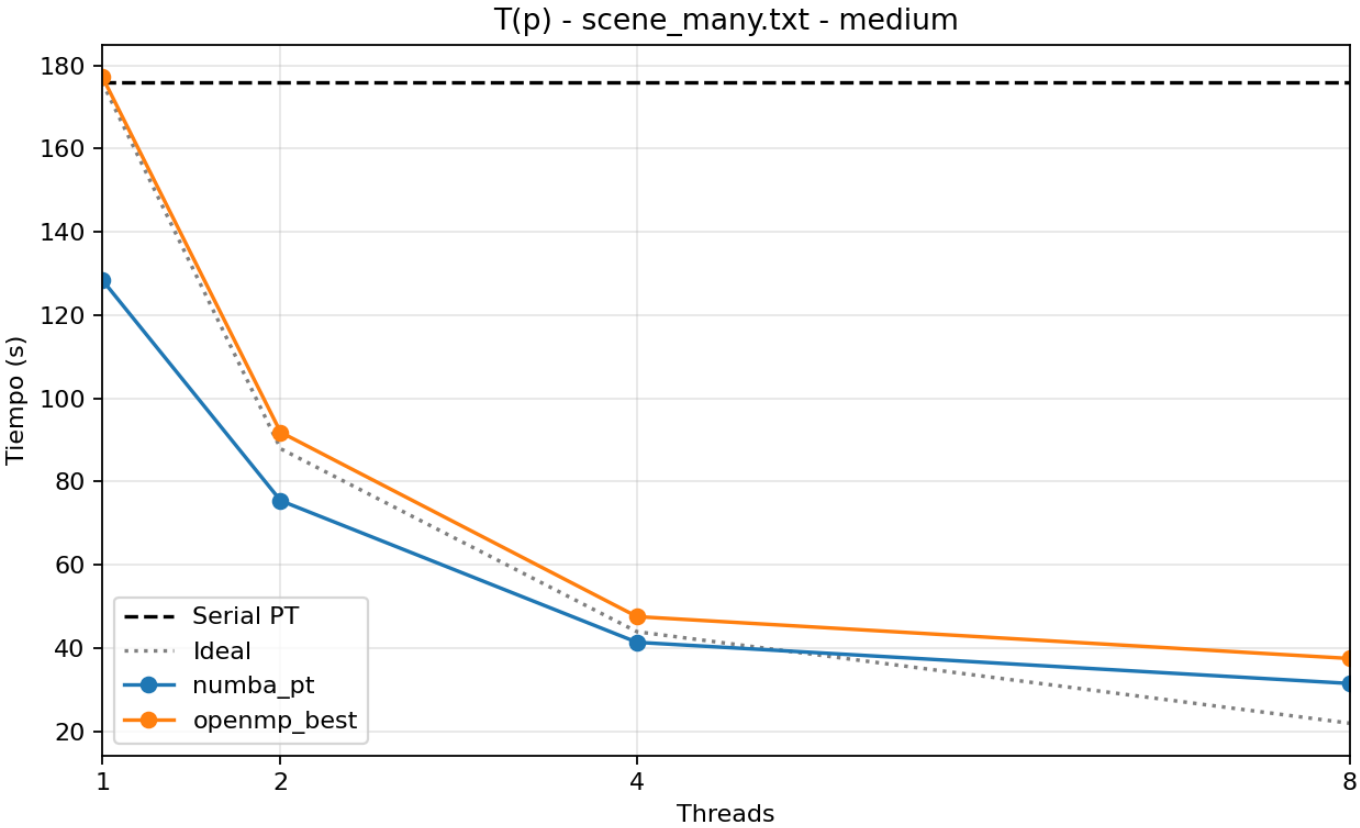

Figure 35: cluster: Tiempo scene_many.txt medium

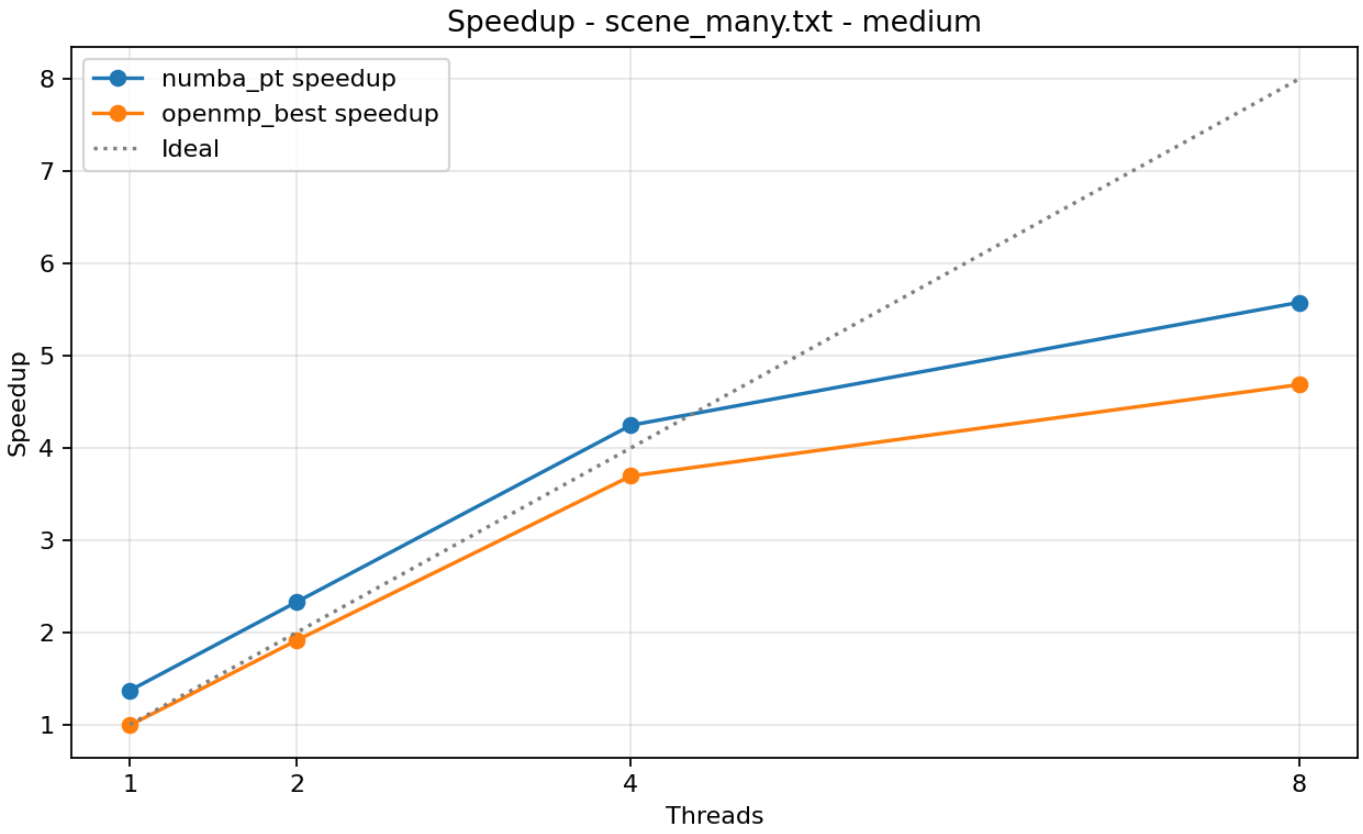

Figure 36: cluster: Speedup scene_many.txt medium

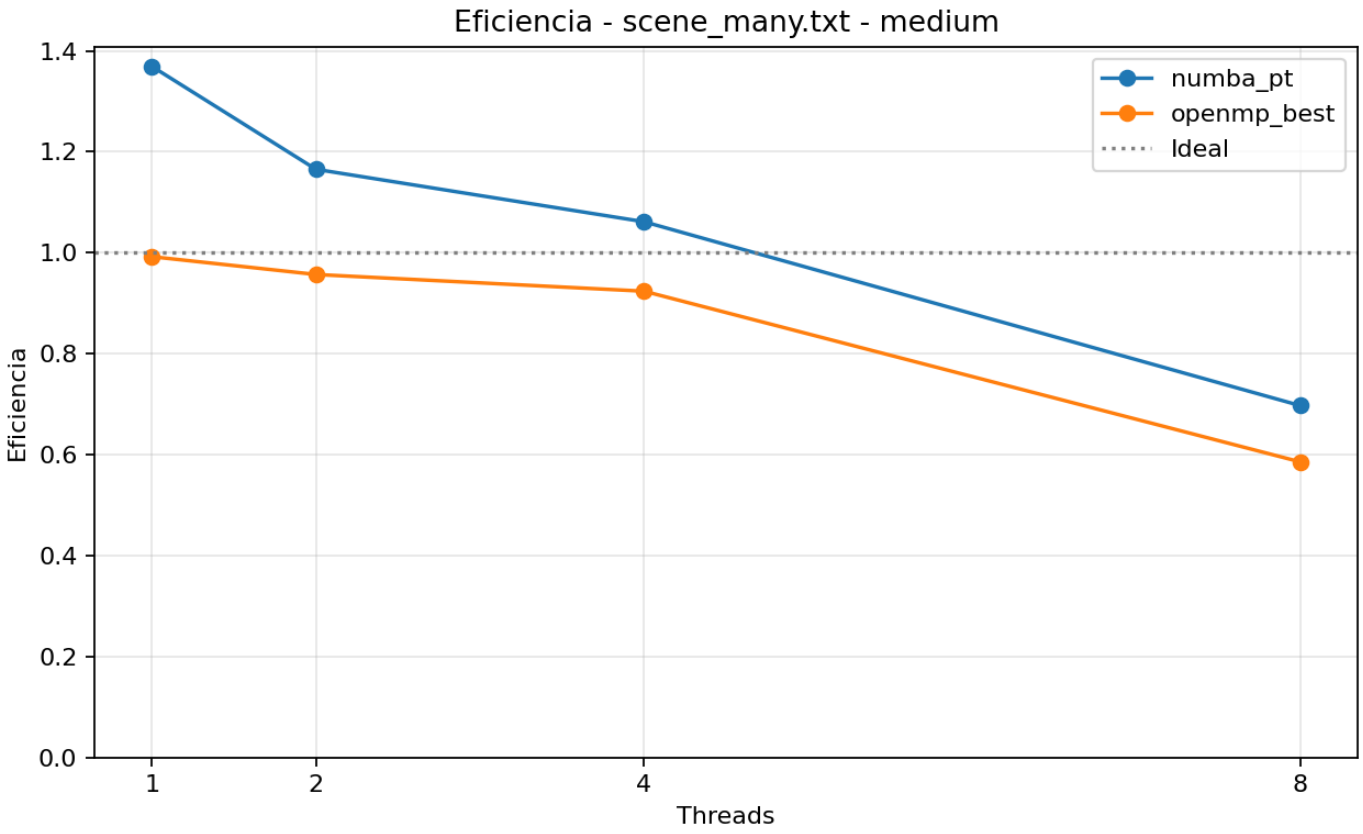

Figure 37: cluster: Eficiencia scene_many.txt medium

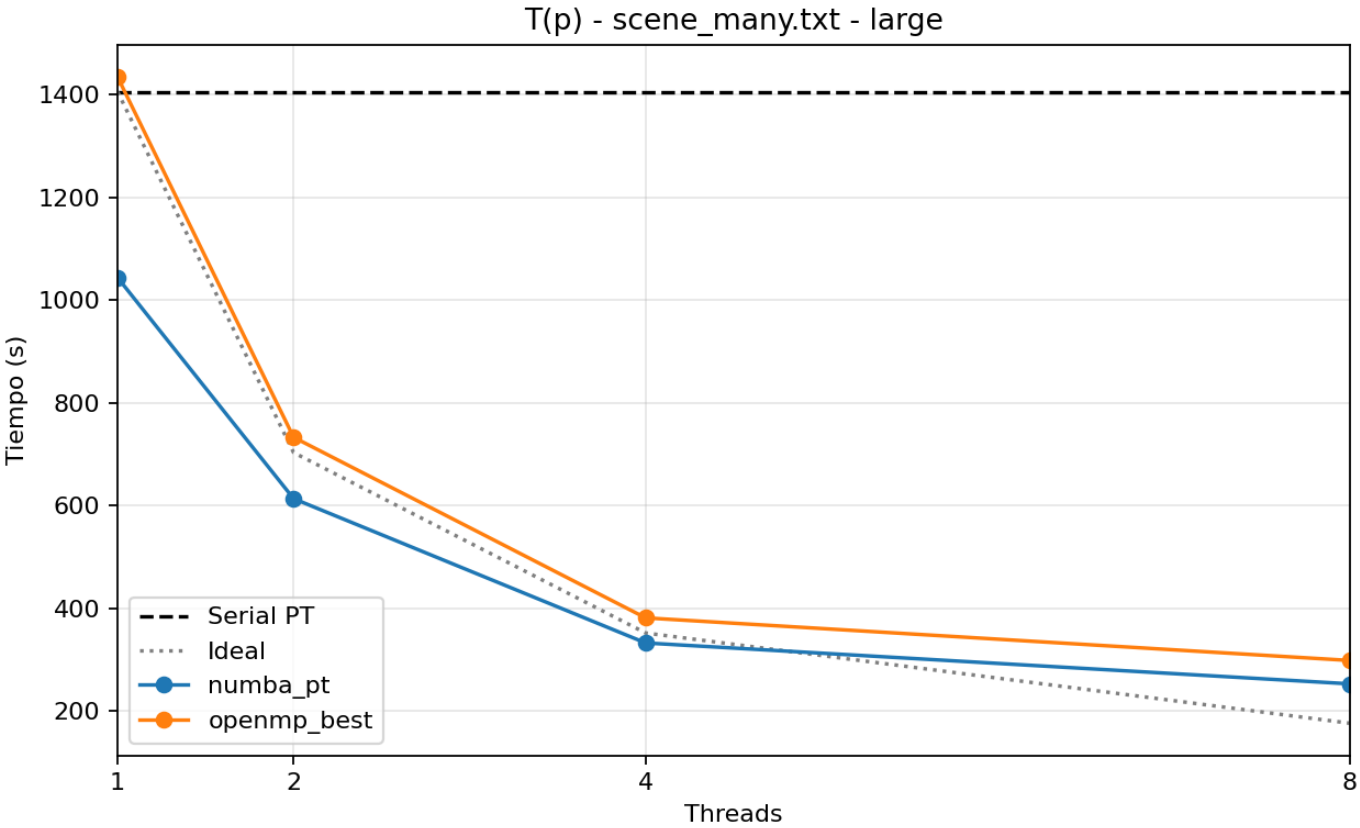

Figure 38: cluster: Tiempo scene_many.txt large

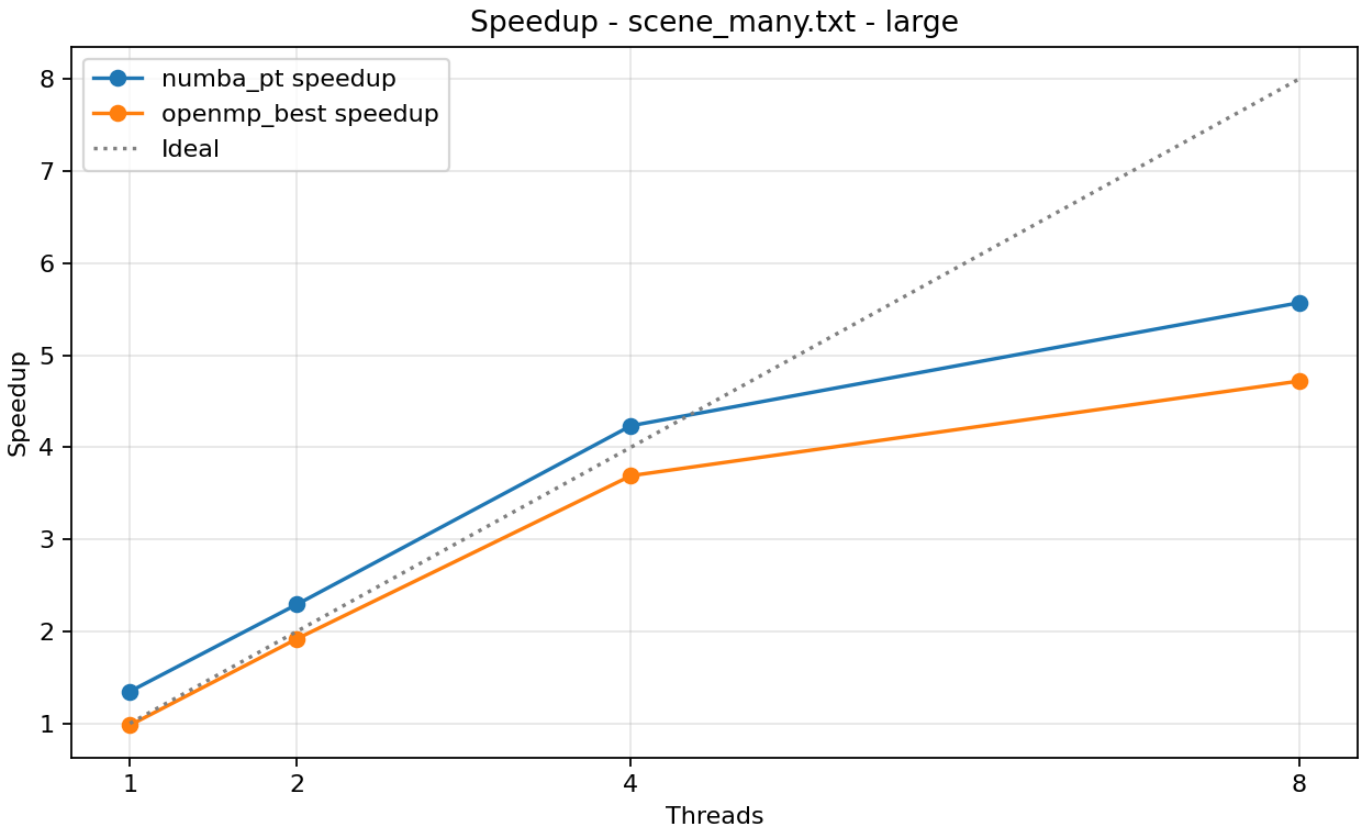

Figure 39: cluster: Speedup scene_many.txt large

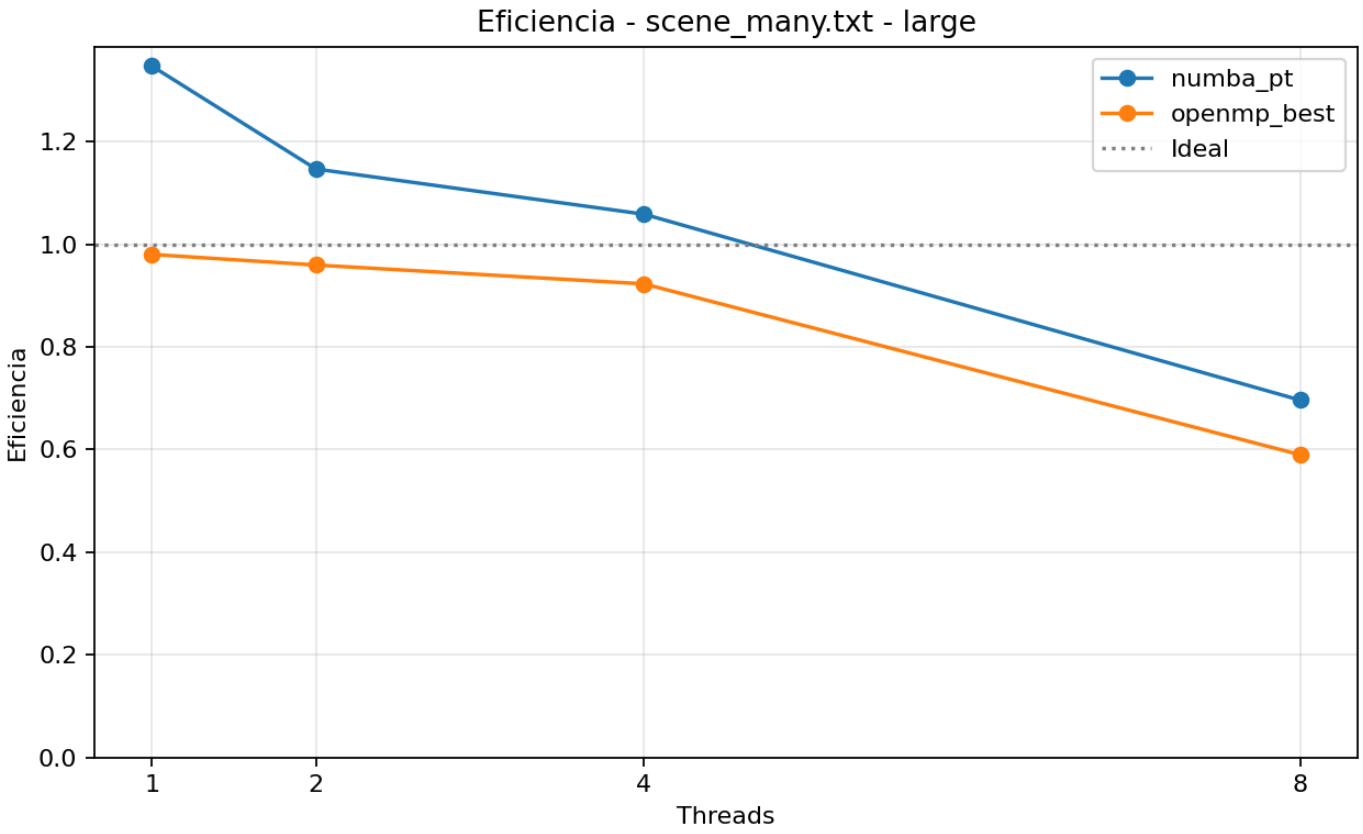

Figure 40: cluster: Eficiencia scene_many.txt large